Machine Unlearning: Taxonomy, Metrics, Applications, Challenges, and Prospects

Na Li, Chunyi Zhou, Yansong Gao, *Senior Member, IEEE*, Hui Chen, Zhi Zhang, *Member, IEEE*, Boyu Kuang, and Anmin Fu, *Member, IEEE*

***Abstract*— Personal digital data is a critical asset, and governments worldwide have enforced laws and regulations to protect data privacy. Data users have been endowed with the "right to be forgotten" (RTBF) of their data. In the course of machine learning (ML), the forgotten right requires a model provider to delete user data and its subsequent impact on ML models upon user requests. Machine unlearning (MU) emerges to address this, which has garnered ever-increasing attention from both industry and academia. Specifically, MU allows model providers to eliminate the influence of unlearned data without retraining the model from scratch, ensuring the model behaves as if it never encountered this data. While the area has developed rapidly, there is a lack of comprehensive surveys to capture the latest advancements. Recognizing this shortage, we conduct an extensive exploration to map the landscape of MU including the (fine-grained) taxonomy of unlearning algorithms under centralized and distributed settings, debate on approximate unlearning, verification and evaluation metrics, and challenges and solutions across various applications. We also focus on the motivations, challenges, and specific methods for deploying unlearning in large language models (LLMs), as well as the potential attacks targeting unlearning processes. The survey concludes by outlining potential directions for future research, hoping to serve as a beacon for interested scholars.**

***Index Terms*— Data privacy, federated learning (FL), large language model (LLM), machine learning (ML), machine unlearning (MU).**

NOMENCLATURE

Notation	Description
$\mathcal{D}$	Original training dataset.
$\mathcal{D}_u$	Dataset request to be removed.
$\mathcal{D}_r$	Remaining training dataset.
w	Original model parameter.
w_r	Unlearned model parameter.
x_i	Training data points.
$\mathsf{A}(\cdot)$	Machine learning algorithm.
$\mathsf{U}(\cdot)$	MU algorithm.
$\mathsf{A}(\mathcal{D})$	Original trained model.
$\mathsf{A}(\mathcal{D}_r)$	Retrained model.
$\mathsf{U}(\mathcal{D}, \mathcal{D}_u, \mathsf{A}(\mathcal{D}))$	Unlearned model.
y_i	Labels corresponding to x_i.

Received 5 April 2024; revised 14 September 2024 and 7 December 2024; accepted 14 January 2025. Date of publication 11 February 2025; date of current version 6 August 2025. This work was supported in part by thee National Natural Science Foundation of China under Grant 62072239 and Grant 62372236, in part by the Open Foundation of the State Key Laboratory of Integrated Services Networks under Grant ISN24-15, and in part by the Qing Lan Project of Jiangsu Province. *(Corresponding author: Anmin Fu.)*

Na Li and Anmin Fu are with the School of Cyber Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China, and also with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: li_na@njust.edu.cn; fuam@njust.edu.cn).

Chunyi Zhou is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310058, China (e-mail: zhouchunyi@njust.edu.cn).

Yansong Gao and Zhi Zhang are with the Department of Computer Science and Software Engineering, The University of Western Australia, Perth, WA 6009, Australia (e-mail: garrison.gao@uwa.edu.au; zhi.zhang@uwa.edu.au).

Hui Chen is with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: chen_hui@njust.edu.cn).

Boyu Kuang is with the School of Cyber Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: kuang@njust.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2025.3530988

I. INTRODUCTION

DRIVEN by an explosion of data and computational power, deep learning (DL) has showcased stunning performance in various applications such as self-driving [1], [2], predicting a protein's 3-D structure from its amino acid sequence [3], deciphering the genetic code and unveiling the secrets of hidden DNA diseases [4], and the very recent artificial intelligence generated content (AIGC) wave represented by text generation via ChatGPT [5], [6], image and video generation via diffusion model [7], and code generation via Codex [8]. These models are trained on user-contributed data [9]. Unintentionally, this raises privacy concerns as the model memorizes users' private information permanently, which might be leaked through known, i.e., membership inference, property inference, and preference profiling attacks [10], as well as yet disclosed privacy attacks.

Recognizing the importance of protecting user data privacy, national governments have implemented various regulations, with the EU's general data protection regulation (GDPR) [11] being particularly prominent. A key provision of the GDPR is the "right to be forgotten" (RTBF), which empowers individuals to request that data controllers (i.e., model providers) erase personal data and remove any related impacts. Similar rights are enshrined in other jurisdictions, such as Section 1798.105 of the California Consumer Privacy Act (CCPA) and Article 47 of China's Personal Information Protection Law (PIPL), both of which grant individuals the authority to delete their personal data. Moreover, GDPR and PIPL require data controllers to respond to such deletion requests without undue delay. These regulations grant data contributors control over their data even after data release, fostering a willingness to share and contribute high-quality data. This, in turn, benefits (model) service providers by enabling higher service profits and reducing legal risks [12].

In the DL context, traditional data deletion methods, which merely remove raw training data from back-end databases, are insufficient for meeting RTBF requirements. This is because the model still retains the influence of user data through

2162-237X © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

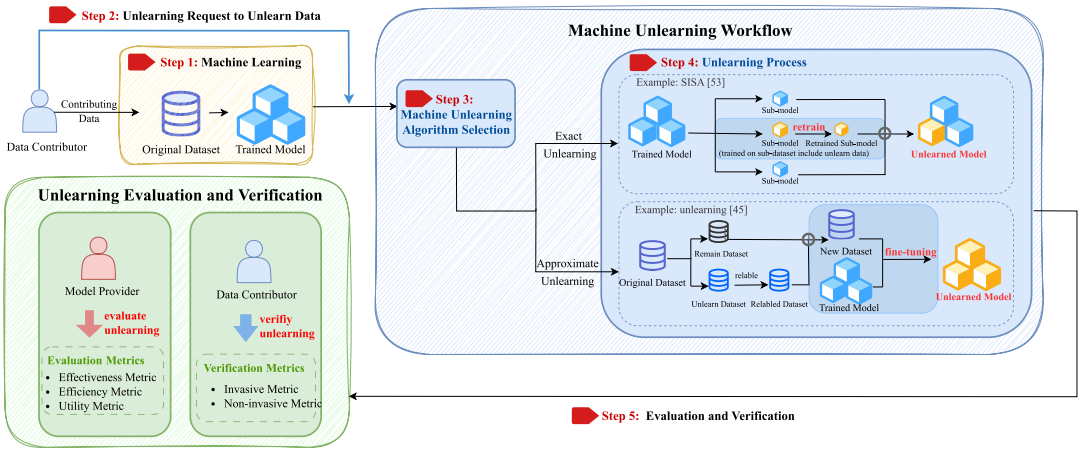


Fig. 1. Lifecycle of MU and two types of unlearning algorithms: exact and approximate unlearning.

(explicitly or implicitly) memorized patterns and features, which can still be exploited by privacy attacks [10], [13], [14]. To address this, a new research direction known as *machine unlearning* (MU) has emerged. Unlike traditional data deletion, which operates at the storage level, MU functions at the model level, swiftly altering the model to remove data and eliminate its influence as if it had never existed, thus ensuring compliance with RTBF while preserving model utility (as depicted in Fig. 1).

Beyond legal compliance, MU has the potential to significantly improve model robustness and ethical alignment. For instance, given that large language models (LLMs) are pretrained on extensive datasets, which may include content misaligned with ethical values—such as biased or unfair data—MU can effectively erase these data, thereby improving model fairness and reducing bias. In addition, MU can also mitigate the harmful effects of adversarial, poisoned, and outdated data, boosting model accuracy and reliability.

The most intuitive and effective method for MU is the retrain from scratch (model retraining), which involves periodically training a new model (retrained model) on the remaining dataset (without the cohort of data to be erased) from scratch. Unfortunately, while this approach ensures that the model has fully erased the requested data, it imposes significant computational overhead and response latency, especially as datasets and models grow, making it impractical or even infeasible for real-world applications such as in the LLMs with tens of billion parameters. To address these limitations, MU leads to the development of two primary categories: exact unlearning and approximate unlearning. Exact unlearning aims to expedite the (re)training process, while approximate unlearning eliminates the need for retraining by fine-tuning the model. Both methods strive to ensure that the unlearned model (model after unlearning) is indistinguishable from the retrained model while significantly decreasing unlearning costs.

A. Contributions of This Survey

MU is undergoing rapid development, but there exists a notable lack of comprehensive summaries and analyses to better depict the state of the art. For instance, there is a deficiency in addressing the challenges of unlearning-enabled applications, as well as a lack of security analyses on MU. To address these gaps, we have conducted a thorough investigation. This survey aims to establish a foundational knowledge base that fosters further research and innovation in MU. The key contributions of this survey are summarized as follows.

1) We conducted a comprehensive review of existing MU algorithms for diverse tasks, systematically categorizing them by unlearning mechanisms, and critically analyzed the merits and limitations inherent to each (sub)category.
2) We provided an in-depth analysis of the challenges in distributed unlearning (DU) settings, categorizing methods and comparing their respective advantages and disadvantages.
3) We developed a taxonomy of existing verification and evaluation metrics in MU, designed to assist both data and model owners by emphasizing the primary focus of each metric.
4) We highlighted the diverse applications of MU, with a particular emphasis on the motivations, challenges, and current research methods for deploying unlearning in LLMs.
5) We conducted a thorough examination of the challenges associated with MU, outlining potential research directions for future scholars to explore and reference.

B. Comparison With Existing Surveys

Table I comprehensively summarizes the differences between our survey and previous surveys in all key dimensions. The primary distinctions are evident in several aspects. First, we clearly distinguish verification and evaluation metrics, which are vital in paving the way for the practical implementation of MU. In contrast, many existing surveys confuse these metrics or even overlook them, as seen in the work of Shaik et al. [21] considering only the evaluation metric. Second, we provide a detailed summary of the debate on approximate unlearning, a critical perspective missing from other surveys, such as Xu et al. [22]. Third, we delve into the challenges of MU in the distributed setting, analyzing current methods in detail, which is underexplored in prior surveys. Fourth, we point out the difficulties faced by MU in different application scenarios (i.e., LLMs). Previous surveys often lack depth and comprehensiveness in this analysis, such as Romandini et al. [18]. Finally, we categorize various emerging attacks that threaten MU, elaborating through the lens of

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH CURRENT SURVEYS

Paper	Year	Verification Metrics		Evaluation Metrics			Taxonomy			Distributed Unlearning		Applications			Attacks against MU				
		Inva.[1]	Non-inva.[2]	Effec.[3]	Effic.[4]	Utili.[5]	Exac.[6]	Appr.[7]	Deba.[8]	Challenges	FU [9]	LLMs[10]	RES[11]	IoT[12]	MIA[13]	DPA[14]	MURA[15]	DRA[16]	JA[17]
[15]	2021	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–
[16]	2022	–	–	–	–	–	✓	✓	–	–	✓	✓	✓	–	✓	✓	–	–	–
[17]	2023	–	–	✓	✓	–	✓	✓	–	–	–	–	–	–	–	✓	–	–	–
[18]	2024	–	–	–	–	–	–	–	–	✓	✓	–	–	–	–	–	–	–	–
[19]	2024	–	–	–	–	–	✓	✓	–	–	–	–	–	–	–	–	–	–	–
[20]	2024	–	–	–	–	–	✓	✓	–	–	–	–	–	–	–	–	–	–	–
[21]	2024	–	–	✓	–	–	–	–	–	–	✓	–	–	–	–	–	–	–	–
[22]	2024	–	–	✓	✓	–	✓	✓	–	✓	✓	–	–	–	✓	✓	✓	✓	–
Ours	**2024**	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

(1) Inva. – Invasive Metric (2) Non-inva. – Non-invasive Metric (3) Effec. – Effectiveness Metric (4) Effic. – Efficiency Metric (5) Utili. – Utility Metric (6) Exac. – Exact Unlearning (7) Appr. – Approximate Unlearning (8) Deba. – Debate on Approximate Unlearning (9) FU – Federated Unlearning (10) LLMs – Unlearning for Large Language Models (11) RES – Unlearning for Recommender Systems (12) IoT – Unlearning for Internet of Things (13) MIA – MU-specific Membership Inference Attack (14) DPA – MU-specific Data Poisoning Attack (15) MURA – Malicious Unlearning Request Attack (16) DRA – MU-specific Data Reconstruction Attack (17) JA – MU-specific Jailbreak Attack

their adversarial objectives. This facet is often addressed only partially in most surveys, merely offering analyses of a select subset of attack methodologies.

II. BACKGROUND

This section will furnish a comprehensive introduction to MU, encompassing its definition, the requisite properties that algorithms need to fulfill, operational workflow, and various unlearning requests.

A. Definition

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ represent the original training dataset, where x_i denotes the ith training data point, and y_i denotes the corresponding label. The learning algorithm $\mathsf{A}(\cdot)$ generates a model $\mathsf{A}(\mathcal{D})$ trained on $\mathcal{D}$, referred to as the original model. Let $\mathcal{D}_u \subset \mathcal{D}$ be the subset requested for removal, with its complement $\mathcal{D}_r$ (where $\mathcal{D}_r \cap \mathcal{D}_u = \emptyset$ and $\mathcal{D}_r \cup \mathcal{D}_u = \mathcal{D}$) representing the remaining data. The function $\mathsf{U}(\cdot)$ denotes the MU algorithm.

When an unlearning request is initiated, it is crucial to remove all information related to $\mathcal{D}_u$ from the trained model $\mathsf{A}(\mathcal{D})$. This is achieved by executing the unlearning algorithm $\mathsf{U}(\cdot)$, which constructs an unlearned model denoted as $\mathsf{U}(\mathcal{D}, \mathcal{D}_u, \mathsf{A}(\mathcal{D}))$. The unlearned model should be indistinguishable from the retrained model $\mathsf{A}(\mathcal{D}_r)$ [12], which is obtained through naive retraining from scratch and contains no information about $\mathcal{D}_u$.

B. Properties

A well-designed MU algorithm should fulfill the following three properties.

1) *Effectiveness:* Ideally, the unlearned model should eliminate any information associated with the forgotten data, making it as if the model had never been exposed to it. However, in practice, a minimal amount of residual information that remains in the model which cannot be exploited by attackers is considered acceptable.
2) *Efficiency:* The unlearning needs to respond promptly to user requests and be completed within the legally required time frame. In this context, the unlearning algorithm should be computationally cost-efficient.
3) *Utility:* Post unlearning, the utility, i.e., the accuracy of the model on the remaining dataset, should be consistent with its capability before unlearning. The unlearning process should not impair the utility of the model.

C. Workflow

The general workflow of MU is illustrated in Fig. 1. In the context of machine learning as a service (MLaaS), a model has been trained on an original dataset to provide high-performance services. Due to privacy and security concerns, data contributors may want to withdraw his/her personal data (or specific data features, etc.) from the deployed model [12]. Consequently, data contributors initiate an unlearning request to the model provider (the server), requiring the model to effectively forget the corresponding data [23]. Upon receiving the unlearning request, the server selects an appropriate unlearning algorithm (i.e., subcategorical methods in exact unlearning or approximate unlearning) to execute an unlearning based on specific factors such as unlearning data type (i.e., text, image, and graph), model type (i.e., convolutional neural networks (CNNs), deep neural networks (DNNs), graph neural networks (GNNs), and linear models), and requirements prioritizing either thorough deletion or speed [24], [25]. Then, the server obtains an unlearned model that does not contain any information about the forgotten data.

D. Unlearning Request

Given the diversity of ML applications, unlearning demands differ in practice. Next, we will discuss four distinct types of unlearning requests.

1) *Item Removal:* Requests to eliminate the influence of specific items or samples from models are the most common in MU. For example, in recommendation systems, items typically refer to individual products or user interactions [26]. Similarly, in graph embedding, specific nodes or edges can be seen as items that may need to be unlearned. Furthermore, in natural language processing, sentences or even whole documents are often treated as items, given that individual tokens generally lack practical meaning [27].
2) *Feature (Attribute, Concept, or Style) Removal:* In real-world scenarios, models often need to forget all information related to a specific subject (i.e., feature, attribute, concept, or style). For example, in credit assessment models that operate on tabular data, it may be essential to unlearn specific features like religious beliefs to mitigate bias. Similarly, Guo et al. [28] explored the removal of certain attributes, such as smiles, from image data in vision models. Additionally, in LLMs, Eldan and Russinovich [29] attempted to make Llama2-7B [30]

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

forget concepts about Harry Potter. Furthermore, in text-to-image models, protecting the copyright of an artist's style is crucial; if infringement occurs, the model must be made to forget that specific style [31].

3) *Class Removal:* This type of request involves erasing information associated with one or more classes from a trained model. Such requests are particularly common in classification tasks; for instance, in face recognition, where each class represents an individual's face. When a user withdraws from the system, the model must eliminate all information related to that specific class. Additionally, in generative tasks, each class typically represents a word or token within the vocabulary.
4) *Task Removal:* Nowadays, ML models are increasingly trained for multiple tasks rather than a single one [32]. This is particularly evident in the rise of LLMs, which are initially pretrained on generic tasks and later fine-tuned for customized downstream tasks such as code generation. However, some tasks are privately customized and require the removal of relevant information once the service is complete. In these cases, it is essential to ensure that the trained (fine-tuned) model can forget certain tasks it has already adapted well.

III. Verification and Evaluation Metric

Upon unlearning, model providers assert that they have removed the influence of forgotten samples $\mathcal{D}_u$ from their models. However, malicious providers may violate this claim for various reasons, such as conserving resources, engaging in data theft, or avoiding performance degradation [33]. To mitigate this risk, verification metrics are essential for data contributors to determine whether the unlearned model still retains unacceptable traces of information related to $\mathcal{D}_u$. As discussed in Section II, idea MU algorithms should satisfy the properties of effectiveness, efficiency, and utility. Therefore, model providers require evaluation metrics that align with these properties to assess their unlearning algorithms, while data contributors need verification metrics to ensure the genuine removal of data.

A. Verification Metrics

According to whether the verification impacts the model training, it is classified into invasive and noninvasive metrics.

1) Invasive Metric: Invasive metrics involve data contributors actively injecting specific markers during model training, and verifying the success of unlearning by assessing the performance of the unlearned model on these markers.

a) Watermark-based metric: The process consists of two key steps: watermarking and verification [34]. During watermarking, data contributors typically embed predetermined patterns (i.e., backdoor triggers and owner-specific binary strings) into the data (i.e., $\mathcal{D}_u$, model parameters) as watermarks, or generate adversarial examples for the same purpose. In the verification step, they analyze the model's response to these watermarks immediately after model providers claim that unlearning is complete. If the target data is not deleted, the model will likely exhibit the predetermined pattern in response to the watermarks.

These watermarks serve as critical verification tools as they provide tangible and distinct changes before and after unlearning. However, there are significant limitations. First, injecting watermarks raises security risks, especially for other data contributors [35]. Second, in data outsourcing scenarios, training-based defenses [36] are likely to remove the watermark effect, making the verification fail [35].

1) *Backdoor-Based Watermark:* Sommer et al. [37] conducted a verification through the backdoor. The process begins with the watermarking step, in which data contributors craft backdoor patterns that alter the predictions of $\mathcal{D}_u$ to an artificial target label. These poisoned samples (watermarks) are then uploaded to the server. During the verification step, once the model has completed the unlearning, data contributors evaluate the statistics on the backdoor success rate. If the model retains the relevant information of $\mathcal{D}_u$, the backdoor success rate will be notably high. Gao et al. [34] extended [37] to unlearning verification under FL settings. However, [34] and [37] just randomly select pixels and set their values of 1 as the backdoor trigger, which can be easily detected by model providers. To counteract this, Guo et al. [38] used the least significant bit algorithm to inject triggers into watermarks.
2) *Feature-Based Watermark:* Izzo et al. [39] proposed the feature injection test (FIT) mainly for the unlearning verification of linear classifiers. The watermarking step is where they inject a strong signal into $\mathcal{D}_u$, involving the addition of an extra feature which is set to zero. The training process will assign a significantly different weight from zero to this feature. The verification step is that, because unlearning will cause the weight on this feature to become zero again, so unlearning can be verified by detecting the change of the unlearned model's weight on this special feature before and after deletion. The closer the change is to zero the more successful the unlearning is.
3) *Adversarial Example-Based Watermark:* Gao et al. [34] introduced the first verification metric under FL, watermarking the local model of a particular user to verify that his entire data is removed from the global model. Since the unique nature of a DNN classifier can be represented by its classification boundary [40], they leveraged the boundary fingerprinting [40] to find decision boundary fingerprints as watermarks for uniquely identifying the local model. Specifically, the process begins with a watermarking phase, where data contributors generate adversarial examples (near the decision boundary) to assess the robustness of the local model as watermarks. Then fine-tune the local model to obtain a watermarked version that demonstrates a more smoothed (robust) boundary around the watermarks. This watermarked local model is subsequently transmitted to the server and aggregated into the global model. In the final verification step, the prediction results of the global model on the adversarial watermarks are checked. Successful unlearning is indicated by the rapid erosion of the smoothed boundary around the watermarks, leading to incorrect predictions by the global model.

2) Noninvasive Metric: A common scenario is that data contributors do not actively watermark data in advance to verify model providers' claimed authenticity—i.e., in legacy models. In this case, data contributors can adopt noninvasive metrics, which utilize the model's output or require model providers to offer cryptographic proof to verify the unlearning. Below, we summarize existing noninvasive metrics.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

a) Membership inference metric: Membership inference attacks can determine whether a given data sample exists in the training dataset [41], [42]. Notably, if an attack still identifies the unlearned data as a member after it is purportedly forgotten, it signals a failure in the unlearning process.

Specifically, Graves et al. [43] initially trained a shadow model to mimic the unlearned model. Subsequently, they used the shadow model's output (i.e., confidence scores) on both the training and nontraining data as meta-data to train a binary meta-classifier. This meta-classifier then determined the membership of $\mathcal{D}_u$, such that an output of 1 signified the retained influence of $\mathcal{D}_u$ on the model. Ma et al. [9], on the other hand, introduced the forgetting rate (FR), calculated by comparing the precision of membership inference before and after unlearning. The FR intuitively gauges the success of the unlearning process by tracking the transition of $\mathcal{D}_u$ from member to nonmember status. An FR value of 1 demonstrates complete unlearning.

b) Data reconstruct metric: Data reconstruction can reverse a model's training data information upon model outputs (in black box) or parameters (in white box) [43], [44]. Therefore, if an attacker reconstructs the unlearned data or its labels from the unlearned model, it demonstrates a failure in the forgetting process.

In [43] and [45], the unlearning is conducted at class level rather than item level. To verify class-level data removal, they adopted a modified version of the threat model proposed by [46] to reconstruct the data's class information. After initializing the input vector with zeros, a small amount of noise is injected. The threat model is optimized by gradient descent using the loss calculated concerning this input and the target class (forgotten class). After every n steps of gradient descent, an image processing step is performed, which helps in the recognition of the generated images. The whole process is repeated for a certain number of epochs to get the final inverted images. If the unlearning has been done correctly, the data reconstructed from the unlearned model's parameters should not contain any information about the forgotten class. Salem et al. [47] utilized the change in the output of a black-box model before and after unlearning to carry out an adversary attack that could reconstruct the unlearned sample to verify sample-level data removal.

c) Cryptographic-based metric: If model providers can offer cryptographic proof of unlearning, data contributors will have substantial reasons to believe that data erasure has been successful.

As most existing verification lack theoretical guarantees, Eisenhofer et al. [48] constructed the first protocol for verifiable unlearning. This instantiation protocol uses succinct noninteractive argument of knowledge-based verifiable computation for proving model updates induced by unlearning, and hash chains for proving nonmembers in a training dataset. Model providers can use this protocol to provide cryptographic proof that an agreed-upon unlearning process has been executed.

B. Verification Framework

The previously discussed unlearning verification metrics only indirectly verify the effectiveness of unlearning techniques. In addition, unlearning requests typically involve multiple data points, but these metrics only qualitatively determine whether the server has deleted specific data. They cannot quantitatively verify the extent of unlearning, such as precisely estimating the volume of unlearned data. Therefore, they are merely indicators rather than verification frameworks.

Next, we introduce a recent attempt toward a standard framework for unlearning verification, namely TruVR [35], which aims to utilize a holistic toolkit to address dishonest servers in class-level MU. It is important to note that, as of now, there is no universally accepted standardized verification framework, and existing methods are largely empirical including TruVR. TruVR begins by defining three types of dishonest model providers (servers).

1) *Neglecting Server:* The server ignores the unlearning request and retains the original model unchanged.
2) *Lazy Server:* The server selectively unlearns only part of the requested data.
3) *Deceiving Server:* The server unlearns irrelevant samples of the same class and volume as the unlearning request.

Different verification methods are designed to verify different kinds of unlearning deceptions.

1) *Unlearned Class Verification:* By extracting model sensitivity from both the original and unlearned models, the data provider can use comparative statistical analysis to verify if a target class has indeed been unlearned by the server, which is particularly effective in identifying the neglecting server.
2) *Unlearned Volume Verification:* The data provider trains multiple shadow models on incrementally divided target data and extracts the model sensitivity of each model. By learning the function between model sensitivity and unlearned data volume, the data provider can infer the volume of unlearned data by the server, effectively identifying the lazy server.
3) *Unlearned Sample Verification:* When the server tricks the data provider by retaining target data in the unlearned model, the distinguishable gap between the two model sensitivities (original and unlearned model) can reveal the presence of data. This allows the data provider to verify whether the server has honestly unlearned the requested data, effectively identifying the deceiving server.

C. Evaluation Metrics

Evaluation metrics enable model providers to assess the effectiveness, utility, and efficiency of their unlearning processes, allowing for better optimization of unlearning algorithms.

1) Effectiveness Metric: Unlearning effectiveness means that the unlearned model should perform as if it had never seen $\mathcal{D}_u$. By evaluating the effectiveness, model providers can measure the quality of the model's unlearning capabilities.

a) Relearn time metric: The relearn time is the number of epochs required for the unlearned model to regain the same accuracy on $\mathcal{D}_u$ as the original model. It indirectly measures the amount of information residing in the unlearned model. If the relearn time is small, there is a high probability that the unlearned model retains more information. Conversely, the unlearning is effective [49], [50]. Chundawat et al. [45] indicated that the unlearned model can rapidly regain significant accuracy but may not converge to the original accuracy on $\mathcal{D}_u$ for a long period. Hence, determining relearn time solely based on reaching or surpassing the original accuracy in epoch number can be misleading. To counter this,

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

an $\alpha\%$ margin around the original accuracy is introduced to calculate the anamnesis index (AIN) [45]. AIN approaching 1 indicates effective unlearning, while values much lower than 1 suggest there is information left, corresponding to a shorter relearn time. AIN substantially exceeding 1 may, however, signify notable parameter changes, resulting in over-unlearning, potentially causing a "Streisand effect" (private data is more accessible to the adversary).

b) Similarity-based metric: The similarity between the unlearned model and the retrained model is illustrated by measuring the distance between them in terms of activation, weight, and distribution (the guarantee of the unlearning effectiveness provided becomes stronger in the above order). The higher the similarity, the better the unlearning.

1) *Activation Distance:* Activation similarity provides a weaker evaluation of unlearning. In [49] and [58], the L_1-norm distance is utilized to quantify the similarity of the final activations between the unlearned and retrained model. The smaller the L_1-norm is, the higher the unlearning effectiveness.
2) *Weight Distance:* As the modification of model weights is a common operation in most approximate unlearning processes, various similarity metrics are used to verify the correlation of weights between unlearned and retrained models. The common metrics are L_2-norm distance and cosine similarity. The smaller the L_2-norm distance or cosine similarity is, the better the unlearning effectiveness [39], [43], [56]. Note that the cosine similarity is limited to the evaluation of classification tasks. Although these metrics are simple, they impose practical limitations such as the necessity to retrain the model from scratch and the potential deviation due to floating point operations. To counter this, an extended version of the L_2-norm distance metric for verifying approximate unlearning was proposed by Thudi et al. [59]. They eliminate the necessity for naive retraining by calculating the L_2-norm distance between the final approximate unlearning weight and the initial weight, significantly reducing computational costs.
3) *Distribution Distance:* Ensuring distribution similarity between the unlearned and retrained model provides a higher guarantee of unlearning efficacy, while measuring such similarity is nontrivial. The KL divergence is a common metric used to measure the distribution distance between two models: the closer to 0, the better. Chundawat et al. [65] used Jensen–Shannon (JS) divergence to calculate zero retrain forgetting, which compares the output distribution before and after unlearning.

c) Accuracy on $\mathcal{D}_u$ metric: A well-trained model tends to have good generalization with high accuracy, especially for data in the training dataset. This property can be exploited to indirectly verify the effectiveness of unlearning [56], [57]. For $\mathcal{D}_u$, the ideal accuracy should be the same as a model trained without seeing D_u [25], [60]. Despite being simple and straightforward, this metric lacks sensitivity to the complex and subtlety unlearning process. Nonetheless, the majority of studies have used this metric as the most basic form of evaluating unlearning effectiveness [25], [39], [43], [51], [52], [56], [57], [60], [62].

d) Theory-based metric: Some MU algorithms, i.e., certified unlearning, are inherently designed to have properties that prove their effectiveness.

1) *Retrain-Based:* Exact unlearning is typically aimed at rapidly retraining with $\mathcal{D}_r$, which ensures that the distribution of an unlearned model and a naive retrained model are identical. For example, in [12] and [51], when unlearning $\mathcal{D}_u$ is requested, relevant submodels that learned on $\mathcal{D}_u$ are retrained, thereby ensuring complete forgetting. Note that this metric is inapplicable for approximate unlearning.
2) *Certified-Based:* Certified unlearning was first proposed by Guo et al. [52], which adds delicate noise to the weights [49], [53], [54], [55], [56] or the loss function [52] based on differential privacy (DP). This guarantees that the outputs of the unlearned model are indistinguishable from the retrained model. However, this metric does not apply to models with nonconvex loss functions that are common in DL.

2) Efficiency Metric: The efficiency of the unlearning process also needs to be verified, because RTBF requires the service provider to complete the unlearning operation within a specified time. The faster it is completed, the sooner the service can continue to be provided to users.

a) Unlearn speed metric: Unlearn speed (running time) can be used to assess the unlearning efficiency, measuring the time difference between unlearning and naive retraining. The larger the difference, the faster the system restores privacy, security, and usability [12]. It can be observed from Table II that the majority of studies utilize this metric to evaluate the unlearning efficiency.

3) Utility Metric: Utility means that the performance of the model on $\mathcal{D}_r$ should be consistent before and after unlearning. Removing data from the trained model may deteriorate its performance, which is undesirable for normal cases. Therefore, it is imperative to evaluate the utility of the unlearned model to ensure it is usable.

a) Accuracy on $\mathcal{D}_r$ metric: For $\mathcal{D}_r$ or test dataset, the ideal accuracy should be the same as a model trained without unlearned. This metric can be used to evaluate the influence of unlearning on remaining data and verify whether the utility of the unlearned model is adversely impacted.

b) Summary: Verification metrics allow data contributors to verify that the unlearned model does not contain any traces of forgotten data. Invasive verification metrics are relatively easy to implement with higher accuracy. However, they rely on watermarked data beforehand that can intrude into the model training process, which may pose a security risk (i.e., backdoor-based watermarks could be maliciously exploited). Noninvasive verification metrics do not impact the model as they only passively use the model's output information, but the verification process is relatively complex, especially for membership inference where an additional binary meta-classifier needs to be trained (i.e., could be costly).

Evaluation metrics facilitate model providers to assess the effectiveness, efficiency, and utility of the MU algorithm, and to determine whether to publish an unlearned model based on the evaluation. However, some metrics are simple and only coarsely measurable, i.e., based on accuracy, while some metrics are limited in their application, i.e., based on theory. Notably, many metrics are based on a comparison with the retrained model, i.e., metric based on similarity, which is not practical to obtain the parameters of the retrained model as a reference.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUMMARY OF METRICS

Metrics			Advantages	Drawbacks
Verification Metrics	Invasive Metrics	Watermark-based [34], [37]–[39]	Conclusion can be obtained intuitively	Damage model's performance
	Non-invasive Metrics	Data reconstruct [43]–[47]	Conclusion can be obtained intuitively	The implementation is complex
		Cryptographic-based [48]	Theoretical guarantees for verification	Difficult for users to understand
		Membership inference [41]–[43]	Inferences can take a variety of ways	The implementation is complex
Evaluation Metrics	Unlearning Effectiveness	Relearn time [45], [49], [50]	Evaluation processes easy to understand	Evaluation capability is weak
		Theory-based [12], [51], [52] [49], [53]–[57]	Theoretical guarantees for evaluation	Only for specific unlearning method
		Similarity-based [39], [43], [45], [49], [56], [58], [59]	Evaluation process is relatively rapid	Need to compare with retrained model
		Accuracy on $\mathcal{D}_u$ [25], [39], [52], [56], [57] [43], [51], [60]–[62]	Evaluation processes easy to implement	The capability for evaluation is weak
	Unlearning Efficiency	Unlearn speed [12], [25], [56], [57], [60] [39], [43], [62]–[64]	Evaluation processes easy to understand	Need to compare with retrained model
	Model Utility	Accuracy on $\mathcal{D}_r$ [12], [43]	Evaluation processes easy to implement	Evaluation results are not convincing

TABLE III
COMPARISON OF EXACT AND APPROXIMATE UNLEARNING

Category	Basic goal	Advantages	Drawbacks
Exact Unlearning	Aligning the unlearned model's distribution with that of a retrained model	More cleaner and effectiveness unlearning	Expensive time and computational overhead, Hard to implement in complex models, Hard to design exact unlearning algorithms
Approximate Unlearning	Indistinguishable parameters between the unlearned and retrained model	Applicable to more complex models, Requires minimal time investment, Easier to achieve than exact unlearning	Maintains sensitive information in parameter

IV. MU TAXONOMY

In centralized machine learning (ML), unlearning algorithms aim to balance three critical objectives: ensuring effective forgetting of $\mathcal{D}_u$, maintaining utility on $\mathcal{D}_r$, and minimizing computational costs. These algorithms are classified into two categories based on whether they fully eliminate the influence of forgotten data: exact unlearning and approximate unlearning. To facilitate a clearer understanding, we define each category and provide a detailed explanation of their distinctions.

Definition 1 (Exact Unlearning): Given a learning algorithm $\mathsf{A}(\cdot)$, for any subset $\mathcal{T} \subseteq \mathcal{U}$, the algorithm $\mathsf{U}(\cdot)$ is considered an exact unlearning algorithm if, under the same random seed, it satisfies

$$P(\mathsf{A}(D_r) \in \mathcal{T}) = \mathrm{P}(\mathsf{U}(D, D_u, \mathsf{A}(D)) \in \mathcal{T}).$$

where P represents a certain distribution.

From the definition, we can see that exact unlearning aligns the unlearned model's distribution with that of a fully retrained model, ensuring the complete removal of the erased data's influence. This approach inherits and advances the traditional method of retraining from scratch, aiming to accelerate the retraining process [66]. As a representative example, SISA [51] divides the dataset into disjoint shards. Submodels are trained on the corresponding shards and then aggregated to form the final model. Upon unlearning, SISA only needs to retrain the submodel associated with the shard containing $\mathcal{D}_u$, using localized retraining instead of retraining the entire model, which significantly reduces computational costs.

Definition 2 (Approximate Unlearning): Given a learning algorithm $\mathsf{A}(\cdot)$, and $\epsilon > 0$, for any subset $\mathcal{T} \subseteq \mathcal{U}$, the unlearning algorithm $\mathsf{U}(\cdot)$ is considered as an approximate unlearning if it satisfies

$$e^{-\epsilon} \leq \frac{P(\mathsf{U}(D, D_u, \mathsf{A}(D)) \in \mathcal{T})}{P(\mathsf{A}(D_r) \in \mathcal{T}} \leq e^{\epsilon}.$$

From the definition, it is evident that approximate unlearning does not entirely eliminate the influence of erased data in the model, allowing minimal residual information. It can be applied at different levels: the model parameters (ingrained level) or the final activation function (representation level) [67]. Both require statistical indistinguishability from retrained models, although the unlearning guarantee at the representation level is weaker [67]. Typically, approximate unlearning substitutes model retraining with fine-tuning. A representative example is provided in [43], where to forget an entire target class, the labels of all data in that class are randomly reassigned to other classes to create a new dataset. Fine-tuning on this new dataset makes the unlearned model statistically indistinguishable from those of a retrained model.

Exact unlearning often requires retraining submodels or performing complex mathematical computations [12], leading to inefficiencies and limited applicability in complex models and large datasets. However, it typically ensures strong unlearning capabilities and high predictive performance on $\mathcal{D}_r$. In contrast, approximate unlearning employs simpler optimization techniques to better balance unlearning efficacy and efficiency, offering broader applicability, though it may compromise predictive accuracy on the remaining dataset. Table III compares exact and approximate unlearning from multiple perspectives.

A. Exact Unlearning

Due to the computational complexity required for exact unlearning, the initial algorithm primarily targeted conven-

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

tional models based on the convex function, which are more homogeneous in structure and easier to trace the influence of the data. With great innovative efforts in recent years, exact unlearning is applicable to complex models based on the nonconvex function, which can trace the influence of data on interleaved neural networks, i.e., CNNs, DNN, and GNNs. Therefore, the exact unlearning algorithms can be classified into two main categories: conventional model-specific and complex model-specific, depending on whether they can be applied to nonconvex function-based models.

1) Conventional Model Upon Convex Function: Conventional models are typically associated with a convex loss function, which models usually have a specific single model structure, and convex optimization makes the time consumption lower. Early exact unlearning algorithms are designed for such simple conventional models including Bayesian [12], [23], [68], logistic regression [24], and support vector machines (SVMs) [12], [23], [69].

Simply retraining the model from scratch is ideal for achieving exact unlearning; however, it is computationally infeasible. To overcome this challenge, the concept of MU was first proposed by Cao and Yang [12], they converted the learning algorithm into a summation form that follows the statistical query learning. When an unlearning request is received, only the corresponding small number of summations needs to be deleted from the sum, thus reducing the unlearning overhead. However, this method is only suitable for straightforward models (i.e., naive Bayes and SVMs) that can be converted into a summation form. Cao et al. [23] introduced Karma, a causal unlearning method designed to repair damaged ML systems effectively. However, Karma is limited to SVMs and Bayesian-based classifiers. Since 2020, numerous exact unlearning techniques have emerged, each tailored to specific models. For example, Schelter [24] proposed a decremental update method for unlearning in logistic regression. Jose and Simeone [68] developed an algorithm for PAC-Bayesian models that reduces information risk. In addition, Kashef [69] improved the unlearning efficiency for weak nonlinear SVMs through decremental strategies.

2) Complex Model With Nonconvex Function: ML can be translated into optimization problems. Nonconvex loss functions are often used in complex models. However, nonconvex optimization can lead to multiple local optimal solutions, making it difficult to track data and requiring more resources than convex optimization. For example, neural network models use highly nonconvex loss functions due to their multilayer complex structure and nonlinear activation. Achieving exact unlearning in this setting while minimizing time consumption is challenging.

a) CNN or DNN: Ullah et al. [70] devised exact unlearning by storing the model's historical parameters, which applies to empirical risk minimization. Bourtoule et al. [51] introduced a notable method named SISA, which we have discussed previously. However, SISA has lower accuracy compared to a model trained on the entire unsegmented dataset and needs to maintain the whole training data.

b) GNN: Before 2022, MU mainly focused on image and tabular data. Chen et al. [71] introduced GraphEraser based on SISA, the first exact unlearning for GNN. Unlike SISA's random data partitioning, GraphEraser offers two balanced partition methods to preserve graph structural information. However, GraphEraser's high-partitioning time costs limit its applicability to evolving or multigraph settings. To address this, Wang et al. [72] proposed GUIDE, the first model-agnostic inductive graph unlearning algorithm. GUIDE ensures fairness and balance constraints in graph partitioning, outperforming GraphEraser in time efficiency.

B. Approximate Unlearning

More research has focused on approximate unlearning than exact unlearning, due to the former's superior trade-off between unlearning effectiveness and efficiency. Based on the underlying principles of approximation, strategies that focus on manipulating the data are categorized as data-driven approximation, whereas those that directly revise the original trained model are categorized as model-driven approximation. Table IV provides a comparative analysis of various approximate unlearning methods.

1) Data-Driven Approximation: There are two ways to manipulate data: data isolation and data modification. Data isolation can minimize the impact of the data, whereas data modification changes the model's understanding of $\mathcal{D}_u$.

a) Data isolation: Model providers first separate $\mathcal{D}_u$ from $\mathcal{D}_r$, i.e., split the training dataset into several disjoint partitions (subdatasets) like SISA [51]. Submodels are then trained on corresponding subdatasets, and aggregated to provide services. When unlearning is requested, the submodels corresponding to the subdatasets containing $\mathcal{D}_u$ are identified and approximately unlearned. Notably, this method represents an adaptation of SISA for approximate unlearning, where fine-tuning replaces full retraining to obtain unlearned submodels with lower calculation costs.

Neel et al. [54] proposed a gradient-based removal algorithm that performs several gradient descent steps to unlearn the corresponding submodel. The algorithm offers a trade-off between run-time and accuracy for sufficiently high-dimensional data. Gupta et al. [73] introduced streaming unlearning that can handle adaptive sequences of forgetting requests with a strong provable guarantee. Specifically, they employ DP for private aggregation submodels. In [33], the affected submodels will be retrained (not fully retrained), where the retraining termination is based on the trend of residual memory tendency.

b) Data modification: Data modification refers to altering the data information in the training dataset (i.e., modifying the labels corresponding to $\mathcal{D}_u$ [43]), and then performing several iterations of fine-tuning on this new dataset to achieve unlearning.

This process shares certain similarities with data augmentation, which, as discussed in [77], can enhance ML privacy. Both approaches involve dataset modifications, but they differ in method and objective. Data augmentation generates new, similar data points through transformations like rotations to improve model generalization, whereas data modification in unlearning alters the data, such as through relabeling or adding noise, to erase specific information.

Graves et al. [43] proposed amnesiac unlearning, which stores parameter updates about $\mathcal{D}_u$ during training. When an unlearning request comes, $\mathcal{D}_u$ is relabeled with random labels, and the corresponding updates are subtracted from the model parameters. Then unlearning is accomplished by fine-tuning the model for several epochs. However, this process requires significant storage capacity to save parameter updates and may also reduce model performance. Felps et al. [75] introduced a DNN model lifecycle maintenance process that establishes how to handle specific data redaction requests. Similar to [43],

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON OF APPROXIMATE UNLEARNING APPROACHES

Category		Advantages	Drawbacks	Resource Cost	Model Utility
Data-driven Approximation	Data Isolation [54], [73], [33]	Applicable to most model types	Unlearning is not complete	●○○○	●●●○
	Data Modification [43], [45], [50], [74], [75]	Unlearning process is easy to achieve	Impacts the model's utility, Consumes storage resources	●●○○	●●○○
Model-driven Approximation	Influence-based [52] [39], [62], [64], [76]	No need to store additional information	Relatively complex calculation	●●●○	●●●○
	Fisher-based [49], [53], [58]	Maintain original model performance	Relatively high time consumption	●●●○	●●●○
	Distillation-based [42], [65]	More suitable for complex models	Higher computational and time cost	●●●●	●●●○
	Gradient-based [43], [56]	Less computational resource consumption	Large storage consumption, Unlearning is not complete	●○○○	●●○○

unlearning is implemented by poisoning the labels of $\mathcal{D}_u$ within incremental model updates. Tarun et al. [50] combined the noise matrix along with the samples in $\mathcal{D}_r$ and trained the model for one epoch to perform unlearning. The final model demonstrates outstanding performance in unlearning the targeted data classes. Note that [45] extended [50] to the setting where no original training data is available. Chen et al. [74] modified the decision boundary of the original trained model to mimic the decision behavior of the retrained model. They provided two boundary shift methods to unlearn specific classes: boundary shrink and boundary expanding. The former disrupts the decision boundary of the forgetting class by allocating its features to other classes, while the latter disperses the activation of the forgetting class by remapping an additional class assigned to $\mathcal{D}_u$.

2) Model-Driven Approximation: By directly manipulating the model parameters, unlearned models can be indistinguishable from retrained models in their parameter space. This is achieved through a variety of techniques, primarily leveraging influence function [78], [79], Fisher information matrix (FIM) [80], knowledge distillation, and stochastic gradient descent (SGD).

a) Influence function-based: The influence function [78] is used to evaluate the influence of $\mathcal{D}_u$ on the trained model. Then, by updating the model to negate this influence

$$w_r = w + H^{-1}\Delta \tag{1}$$

where H^{-1} is the Hessian matrix for the loss function on $\mathcal{D}_r$, Δ is the gradient for the loss function on $\mathcal{D}_u$, and $B^{-1}\Delta$ is the influence that $\mathcal{D}_u$ has on the model.

Guo et al. [52] used Newton's method to estimate the influence of $\mathcal{D}_u$ on parameters and maximized the elimination of this influence. However, this method is only applicable to linear models. Similarly, Izzo et al. [39] employed the influence function on the convex loss function. Their method improves the runtime efficiency of [52] but is challenging to fit nonconvex models. In contrast, Warnecke et al. [62] shifted the focus of unlearning requests from samples to features and labels. They translate the effects of data into closed-form updates of model parameters—closed form refers to a mathematical expression that provides a direct solution for the specified variables. These updates can be calculated directly without iteration and contribute to the correction of features and labels learned in the model. However, the effectiveness of unlearning decreases as the number of affected features and labels increases.

Traditional influence functions face challenges when applied directly to GNN due to the inherent data dependencies within graphs [64], [76]. To address this issue, Wu et al. [76] proposed the certified edge unlearning, which enables the removal of edges from the model. Similar to [62], they carefully investigated the dependency between data and redefined the unlearning as finding a closed-form update for the model parameters, using the influence function to calculate the update effectively. Furthermore, Wu et al. [64] extended [76] to tasks for graph node unlearning, edge unlearning, and feature unlearning. The core idea revolves around the incorporation of a loss term for the influenced neighbors alongside the conventional influence function, which allows for an efficient and accurate assessment of how parameters respond to a small amount of perturbation in the dataset.

b) FIM-based: When using the influence function or Newton's method to obtain the influence of $\mathcal{D}_r$, the computation of the Hessian matrix can be very large. To improve the efficiency of unlearning, the FIM [80] can be used on $\mathcal{D}_r$ to approximate the Hessian matrix. Meanwhile, optimal noise is injected to unlearn $\mathcal{D}_u$. The unlearned model is given by

$$w_r = w + F^{-1}\Delta + b \tag{2}$$

where F^{-1} is the FIM on D_r, Δ is the gradient about the loss function on D_u, $w + F^{-1}\Delta$ corresponds to the corrective Newton step, and b corresponds to the optimal noise added [81].

Golatkar et al. [53] proposed a robust unlearning algorithm based on the noisy Newton update, which can erase information about a specific class while replacing the Hessian matrix with FIM to improve efficiency. Golatkar et al. [58] generalized [53] to different readout functions. However, the scalability of [53] and [58] decreases as the size of the training dataset increases. The reason is that the computation of the unlearning step exhibits quadratic growth with dataset size. Golatkar et al. [49] solved this problem and introduced an effective unlearning method for mixed-privacy scenarios. This method partitions the training data into core and user data. Core data consists of general information utilized for pretraining and is crucial to retain. User data typically includes the information that users want to be removed. The core weights are learned using a nonconvex algorithm, while the user weights are obtained through strongly convex quadratic optimization. Simply setting the user weights to zero removes the influence of their data. However, as the dataset needs to remain static during the pretraining phase, it may not be suitable for many practical applications.

c) Knowledge distillation-based: Knowledge distillation enables the training of a student model (unlearned model) to selectively mimic the knowledge of a larger teacher model (original model), allowing filtering out sensitive information about $\mathcal{D}_u$ while maintaining the utility of the student model.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

Chundawat et al. [45] employed a bandpass filter to block the flow of sensitive information from the teacher to the student model. However, this method may not be suitable for large-scale models. In a subsequent study [65], they revised [45] by using a pair of (competent/incompetent) teachers to manage the student. The misinformation about D_u from the incompetent teacher is transferred to the student, helping to unlearn samples. While this method is highly efficient, it damages model utility [42]. Kurmanji et al. [42] presented an application-dependent unlearning method based on the teacher–student formulation, which is adaptable to diverse applications.

d) Gradient-based: Gradient-based unlearning approximates the retrained model by correcting the SGD steps.

Graves et al. [43] leveraged gradient descent to trace and exploit data provenance to achieve rapid incremental model updates. However, this method is only applicable to regression models. Furthermore, they proposed DeltaGrad [56], which utilizes Quasi-Newton to eliminate gradients associated with $\mathcal{D}_u$ based on cached intermediate parameters. Nevertheless, it is not well-suited for unlearning a substantial amount of data.

C. Debate on Approximate Unlearning

Recently, some researchers have argued that defining approximate unlearning as generating unlearned models that are indistinguishable from retrained models in parameters space is ill-considered. First, Thudi et al. [82] posited theoretical proof that one can obtain the same model without alterations from training on a pair of nonoverlapping datasets. This implies that attaining a specific location in the parameters' universe is not a sufficient condition for unlearning, essentially questioning the definition of approximate unlearning. Moreover, Tarun et al. [50] argued that utilizing the retrained model parameters as a comparative benchmark for approximate unlearning quality is unreliable, due to the potential existence of numerous parametric configurations capable of efficient unlearning (the retrained model's parameter is only one of them). Even if there is a significant difference in parameters between unlearned and retrained models, it does not necessarily mean that the unlearning process has failed [42]. Therefore, researchers try to achieve approximate unlearning without aiming for indistinguishability, Wang et al. [83] avoided forcing model parameters to conform to a particular distribution. They preserve the differences in distribution between unlearned and retrained models, thereby making it adaptable for more applications (i.e., natural language processing).

Summary: Table V summarizes the main MU algorithms. Exact unlearning proves effective in ensuring comprehensive unlearning while preventing any attempts by adversaries to extract valuable information from the unlearned model. However, its implementation in time or computation is relatively intensive, particularly with DL models. Additionally, the unlearning request occurs repeatedly, rather than being a one-time occurrence, further aggravating resource consumption. Therefore, great efforts have been directed toward approximate unlearning, which offers a favorable trade-off between unlearning performance and efficiency. However, it cannot guarantee complete forgetting.

V. Distributed Unlearning

Under the dual pressure of big data and complex models, large-scale ML training in centralized ML (CML) poses significant challenges in both computational power and storage capacity. To address these challenges, distributed ML (DML) leverages multiple computational nodes in parallel to train ML models. Notably, DML can greatly mitigate privacy information leakage of the user's local data since these data cannot be accessed by others. There are different DML schemes including federated learning (FL), split learning, peer-to-peer, and private aggregation of teacher ensembles.

There still exists privacy leakage risks in DML. For example, FL is vulnerable to exposure of user preferences through preference profiling attacks [10]. Moreover, in many contexts, such as when the data is poisoned or invalid, it is necessary to delete the data locally while unlearning the impact of the data on the global model. Therefore, it is necessary to deploy MU in a DML setting, referred to as DU.

A. Challenges

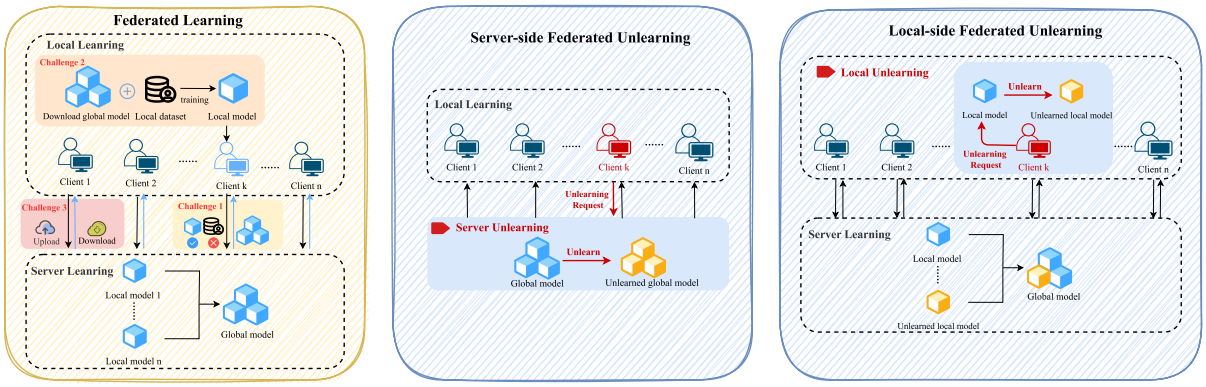
However, most existing MU algorithms are designed under CML and do not apply to DML, primarily owing to the disparate ways they gain information from data. There exist three principal challenges [as shown in Fig. 2(a)]—since current DU algorithms focus on FL settings, we analyze the challenges facing DML mainly taking FL as an example.

1) *Data Availability Perspective:* From the perspective of data availability, three primary issues arise. First, data contributors (clients) are not sharing their data due to privacy concerns, the server does not have access to the local training datasets (i.e., FL). Therefore, server-side retraining becomes virtually impossible, only approximate unlearning can be performed on the server. Second, the frequent drop-in and dropout clients pose a significant challenge for servers attempting to recall former clients for unlearning operations, let alone retraining them from scratch [84]. Third, clients such as edge devices may discard data after local training due to limited storage capacity, so the client may no longer have the same dataset used during the training phase [85].
2) *Model Parameters Perspective:* From the perspective of model parameters, several issues need to be considered. First, training in DML is interactive, i.e., clients continuously share the knowledge learned from the local training dataset with other clients through the global model [86]. Second, updating model parameters in DML is a complex entangled incremental process, i.e., in FL, each local update of each client is based on the updates of all previous clients [86]. Third, DML involves significantly more stochasticity than CML [85], i.e., the clients involved in each training epoch are randomly selected in FL, and the inherent randomness in each client's local training process. Any minor perturbations due to stochasticity could potentially render a domino effect throughout the subsequent training process [85]. The combination of these three issues severely complicates the traceability of data based on global model parameters, compromising the central server's ability to selectively unlearn data from a specific client.
3) *Resource Overhead Perspective:* From the perspective of resource overhead, DML usually has a high communication and time overhead compared to CML due to the necessity for the server to exchange information (i.e., local or global model parameters) with clients. Therefore, ideal MU should not bring additional expensive communication and time consumption to DML.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

TABLE V
SUMMARY OF STUDIES IN UNLEARNING

Category			Papers	Year	Applicable Model	Type of Unlearn Request
Exact Unlearning	Conventional Model upon Convex Function		[12]	2015	Straightforward Learning Models (i.e. naive Bayes)	Samples
			[23]	2018	SVMs and Bayesian-based Classifiers	Subset
			[24]	2020	Logistic Regression	Samples
			[68]	2021	Bayesian	Subset
			[69]	2021	SVMs	Samples
	Complex Model with Non-Convex Function		[70]	2021	Models with Non-convex Functions	Samples
			[57]	2021	Random Forests	Batches
			[25]	2021	Randomized Trees	Samples
			[51]	2021	DNN	Batches, Sequences
			[60]	2022	DNN	Samples
			[71]	2022	GNN	Nodes, Edges
			[72]	2023	GNN	Nodes, Edges
Approximate Unlearning	Data-driven Approximation	Data Isolation	[73]	2020	SGD-Based Models	Samples
			[54]	2021	Convex Models	Non-adaptive Sequences
			[33]	2021	DNN	Samples
		Data Modification	[43]	2021	DNN	Classes, Samples
			[75]	2021	DNN	Batches
			[50]	2021	CNN	Samples
			[74]	2023	DNN	Classes
	Model-driven Approximation	Influence-based	[52]	2020	Linear Models	Samples
			[39]	2021	Logistic and Linear Regression Models	Batches
			[62]	2023	Convex or Non-convex Models	Features, Labels
			[64]	2023	GNN	Nodes, Edges, Features
			[76]	2023	GNN	Edges
		Fisher-based	[53]	2020	DNN	Particular set of training data
			[58]	2020	DNN	Samples
			[49]	2021	DNN	Samples
		Distillation-based	[45]	2023	DNN	Samples, Classes
			[65]	2023	DNN	Classes
			[42]	2023	DNN	Subset, Classes
		Gradient-based	[43]	2020	Regression Models	Subset
			[56]	2020	SGD-Based Models	Samples



(a) (b) (c)

Fig. 2. Challenges in FU and two types of FU workflow. (a) Federated learning and challenges in FU, (b) server-side federated unlearning, and (c) local-side federated unlearning.

To achieve both efficiency and effectiveness, DU must overcome not just the hurdle faced by MU in CML, but also the unique three above-mentioned challenges [87]. Therefore, unlearning in distributed learning is much less explored than that in centralized learning [86].

B. Federated Unlearning

DU primarily focuses on FL, with very little attention given to split learning [88], while other distributed schemes, i.e., peer-to-peer, remain unexplored so far. Therefore, we focus on DU under FL settings.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

FL [89], [90] preserves user privacy by enabling clients to upload only local model parameters, thereby avoiding the need to share raw training data with the server. However, attackers can still infer private information from these parameters, highlighting the need for integrating MU into FL. This integration, termed *federated unlearning* (FU), is vital for enhancing model robustness by unlearning sensitive data or local parameters.

FU can be implemented either server-side, where the server unlearns the global model and redistributes it, or local-side, where clients unlearn the downloaded global model before reuploading it. For example, in healthcare scenarios, FL is applied across multiple hospitals to train a global model without sharing sensitive patient data. If a hospital requests the removal of its data, server-side unlearning approximates the removal since the server lacks access to the original dataset. To ensure complete removal, local-side unlearning can be performed, allowing the hospital to unlearn its downloaded model, though this typically involves higher resource consumption. Next, we will discuss these two methods of FU in detail.

1) Server-Side FU: As local clients never share raw training data with the server during collaborative training, the only way to implement unlearning at the server is to use approximate unlearning on the global model. Therefore, no computational and communication action is required from clients. Fig. 2(b) depicts the overview of server-side FU.

Liu et al. [91] pioneered the study of client-level unlearning, proposing FedEraser to eliminate the contribution of a client on the global model. Specifically, the server maintains all records of historical updates for each client at every FL round. These records are then refined by several rounds of a calibration training process without the forgotten client, accelerating the unlearning process. Nonetheless, this method offers limited improvements compared to retraining from scratch. This is primarily because clients are still required to train the local model to rectify their historical updates, which renders additional rounds of communication between clients and the server. Regarding the issue of FedEraser, Wu et al. [85] emphasized the need to decrease the number of iterations between server and client interactions due to the substantial time and energy consumed by communication, particularly in DNN. Based on this premise, they proposed a solution that directly subtracts the accumulated historical updates from the federated global model parameters and utilizes the knowledge distillation to maintain the model's performance, which effectively eliminates a client's contribution. However, their method requires the server to possess additional outsourced unlabeled data, which may not be feasible in certain high-privacy applications (i.e., medical systems). Zhang et al. [84] applied DP to FU, leveraging clients' historical submissions to eliminate a weighted sum of gradient residuals from the global model. Furthermore, they structured the Gaussian noise such that the unlearned and the retrained model became statistically indistinguishable, effectively removing the influences of individual clients on the global model. Note that [84], [85], and FedEraser have considerable limitations as they focus only on client-level unlearning, thus limiting their practical use when the need is to erase only a subset of the training data, i.e., a specific class. They also require storing historical updates, which creates significant memory overhead, especially for state-of-the-art large models. To address these issues, Wang et al. [81] exploited the CNN channel pruning method to guide the category-level FU. They employed term frequency-inverse document frequency (TF-IDF) [92] to quantify the correlation between channels and categories. The channels with high TF-IDF scores play a more substantial role in distinguishing the forgotten categories and, thus need to be pruned to erase their contributions to the global model.

2) Local-Side FU: Since only approximate unlearning can be performed on the server side, this leads to the preservation of residual sensitive information related to $\mathcal{D}_u$ in the global model. Consequently, some studies suggest that the best unlearning mechanism in FL is to perform retraining among clients [86], [87], [93]. In this case, the unlearning operation must be performed on the local side. However, since the computational resources of edge devices are typically restricted, the primary concern currently is to design a fast unlearning method on the local side that is low cost and maintains the utility of the global model. Fig. 2(c) depicts the overview of local-side FU.

Liu et al. [86] developed a rapid retraining method to fully erase data points from a trained global model. This method employed a diagonal empirical FIM to approximate the Hessian for Quasi-Newton optimization, achieving a low cost while preserving model utility via the momentum technique. Nevertheless, their method is only applicable to models with convex loss functions. Wang et al. [87] proposed an algorithm with parameter self-sharing based on variational Bayesian inference to unlearn specific data points. It can mitigate accuracy degradation caused by unlearning and balance the trade-off between the unlearning effectiveness and model utility. Che et al. [93] introduced the first method for simultaneously implementing the training and unlearning process in FL, based on their previous work in centralized learning [63]. By utilizing [63], a local unlearned model is trained on each local client. Then, leveraging the theory of nonlinear functional analysis to refine the local unlearned model as output functions of a Nemytskii operator, ensuring that the global unlearned model closely parallels the performance of each local unlearned model and significantly speeds up unlearning. Zhu et al. [94] provided a unique heterogeneous knowledge graph embedding unlearning derived from cognitive neuroscience. By combining retroactive interference with passive decay, it erases specific knowledge from local clients and propagates to the global model through knowledge distillation.

Summary: FU conducted on the server side can relatively promptly accomplish the unlearning request without incurring additional communication with the client. However, there are notable shortcomings. First, the unlearning is not complete, with sensitive information remaining in the global model [86], [87], [93]. Second, many current methods are based on information (i.e., historical updates) stored in the server during the training phase [85], [91], while this results in a significant additional storage burden, especially for complex models.

FU performed on the local side can resolve shortcomings faced by the server side. It enables more complete unlearning through retraining and prevents the negative impact of server (model provider) dishonesty, as the unlearning process occurs locally. However, there are also disadvantages, one is that the computational capability of edge devices is insufficient; the other is that effective rapid retraining algorithms are still challenging to achieve.

VI. Application of MU

MU is initially developed to protect user privacy, focusing on basic classification models. Beyond its privacy-preserving

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

capabilities, MU demonstrates significant potential in enhancing model robustness across various application domains (i.e., recommender systems [95], the Internet of Things [96], and LLMs [97]). On the one hand, it effectively mitigates the risks associated with malicious, outdated, or adversarial data. On the other hand, MU serves as a dual-purpose defense mechanism: passively, it cleanses models of the detrimental effects of attacks (i.e., data poisoning and backdoor attacks); actively, it preemptively unlearns sensitive data, thereby disabling privacy threats including membership inference, property inference, and model inversion attacks. Next, we will discuss the application of MU across diverse domains.

A. Unlearning for LLMs

State-of-the-art LLMs have gained significant attention, driving the success of commercial products. From ChatGPT [6] to the rise of DALL-E [98], LLMs have extended their capabilities beyond traditional text-based tasks such as code generation [99]. However, their ability to memorize and reproduce uncensored content raises ethical and legal concerns, including the risk of exposing biased, private, or copyright-protected information. Moreover, LLMs are vulnerable to jailbreaks, or can be misused for malicious activities, such as the creation of chemical, biological, radiological, and nuclear (CBRN) weapons [100]. Given costly training overhead of LLMs, retraining to remove undesirable data effects is unrealistic [101]. MU has emerged as an alternative to eliminate such effects from pretrained (and possibly fine-tuned) models, aligning them with current social norms. While well-studied in classification tasks, MU's application in generative tasks within LLMs remains limited. Thus, in this section, we explore MU in LLMs, termed *LLMs Unlearning*.

1) Motivations: Besides the privacy-related concerns mentioned in classification tasks, particularly to fulfill RTBF, there are additional motivations for LLMs unlearning.

a) Copyright protection: LLMs may be trained on copyrighted content (i.e., books, articles, and other media) during both pretraining and fine-tuning, leading to a myriad of ethical and legal issues as they reproduce and recall these texts. Under Section 106 of the U.S. Copyright Act, copyright holders possess exclusive rights, meaning that unauthorized reproduction or substantially similar expressions potentially infringing. As a result, companies training LLMs are now facing potential lawsuits, scrutiny, and regulatory pressure [101]. For example, The New York Times sued OpenAI and Microsoft, alleging that millions of its articles were used without permission to train AI models like ChatGPT, which replicated content verbatim and mimicked its writing style [102]. In this context, unlearning helps LLMs eliminate copyright-protected content without full retraining. Notably, copyright protection in unlearning is similar to privacy concerns, as both aim to prevent generated text from including specific protected information.

b) Security alignment: LLMs are pretrained on vast, uncurated public datasets, some of which may contain content misaligned with human values, including discriminatory or harmful language related to ethnicity, race, sexual orientation, or gender. Studies indicate that LLMs can generate biased output, such as "men are suited to be engineers, women to be nurses" [27]. The EU's Artificial Intelligence Act mandates supervised AI systems, with enforcement measures for noncompliance to ensure adherence to legal and ethical standards. Thus, aligning AI with human values may necessitate MU to identify and mitigate biased or harmful behaviors. Recent studies demonstrate that compared to the widely used alignment method, reinforcement learning from human feedback (RLHF) [103], unlearning achieves superior alignment performance with reduced computational time [97].

c) Reduce hallucination: Occasionally, LLMs generate seemingly plausible yet deviate from user input, prior generated context, or factual knowledge outputs, known as "hallucinations" [104]. This significantly undermines the reliability of LLMs in real-world applications, where errors—such as fabricated medical diagnoses—can lead to tangible real-life risks [104]. Hallucinations often arise from a combination of factors, such as risky generation strategies and misalignment. By unlearning factually incorrect responses to specific questions, this issue can be effectively mitigated [97].

2) Challenges: The unique characteristics of LLMs present novel challenges for LLMs unlearning.

1) First, the rise of black-box access to LLM-as-a-service complicates the application of traditional unlearning techniques, which typically modify the model's parameters [105]. Furthermore, the inability to alter the model under black-box conditions raises concerns about the feasibility of achieving genuine unlearning.
2) Second, even with white-box access, it is challenging to identify and isolate the influence of unlearning targets (i.e., knowledge concepts and private information) because their effects are widely distributed throughout billions of parameters. Moreover, performing gradient updates for billions of parameters is time-consuming, especially in scenarios requiring frequent unlearning requests.
3) Third, LLMs are trained on vast datasets, yet assuming access to the complete training data for maintaining model utility during unlearning is typically unrealistic due to various constraints, such as data privacy and intellectual property protection [106].

3) Taxonomy: Given the vast number of parameters in LLMs, the efficiency of unlearning operations is critical, as it directly influences the feasibility of implementing unlearning in practice. Accordingly, we classify unlearning approaches from an efficiency perspective into three categories, based on the degree of parameter adaptation required during the unlearning process.

a) Full-parameter unlearning: First, we introduce full-parameter unlearning (FPU), which involves adjusting all trainable parameters during the unlearning process. FPU utilizes two key techniques for parameter updates: gradient ascent (GA) based on first-order information and Fisher information based on second-order information. While this approach offers a strong unlearning guarantee and a better balance between unlearning and model utility, it has a trade-off of significant computational resources and time requirements.

1) *GA-Based:* GA is among the simplest unlearning techniques for classification tasks and has been applied to LLMs unlearning [97], [107], [108]. GA-based unlearning involves updating model parameters by maximizing the loss on the forgot samples, as demonstrated in [107]. However, research indicates that solely relying on GA is insufficient for effectively unlearning undesirable behaviors and may negatively affect the generation capabilities of LLMs [97], [109]. To enhance unlearning effectiveness, Yao et al. [97] minimized a combination of three weighted loss functions: applying GA to the forget

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

samples, enforcing random outputs for those samples, and reducing the KL divergence between the original and unlearned models' predictions on remaining samples. This method is similar to the one in [108]. To better preserve LLMs' generation capabilities, SeUL [109] applied GA to specific sequence spans rather than entire instances. In scenarios where an attacker injects general knowledge into personal data, unlearning such data, as in [107], can weaken the model, whereas SeUL better maintains the model's generation abilities. Eldan and Russinovich [29] noted that GA can harm LLM's language understanding, so they adapted GA into a gradient descent method by minimizing the likelihood of predictions on relabeled forgetting data, such as replacing unlearned text related to the Harry Potter books in Meta's Llama2-7B with generic translations. However, research [110] found that models unlearned through this approach could still generate Harry Potter-related copyright content.

2) *Fisher Information-Based:* Unlearning methods like GA-based, which rely on first-order information (gradients), often introduce significant overheads, such as data preprocessing, or suffer from a lack of robustness [99]. While it is feasible to compute second-order information (Hessian) for shallow models (i.e., logistic regression), the quadratic complexity relative to parameter size renders this approach prohibitively costly for LLMs [99]. However, the potential of second-order information in LLMs unlearning has been underestimated [101]. For instance, Gu et al. [99] showed that adapting inverse empirical Fisher estimation, a Hessian approximation method, to a multi-GPU setting can effectively approximate unlearning. Compared to GA, Fisher-based unlearning offers stronger guarantees of unlearning while better preserving the utility of LLMs.

b) Parameter-efficient Unlearning: Due to the substantial overhead of FPU, parameter-efficient unlearning (PEU) was developed to enhance efficiency by adjusting only specific parameters while keeping the majority frozen. PEU includes two main strategies: modifying a small subset of the existing parameters and introducing extra trainable ones. However, the extent of unlearning achieved with PEU is inferior to FPU.

1) *Existing Parameter Modification.* This approach first identifies and locates the subset of model components (i.e., parameters, weights, or neurons) related to the unlearning target, then optimizes or directly modifies this subset while keeping the rest of the model unchanged. Given the specialized roles of feedforward and attention neurons in LLMs, certain neurons harboring unlearning target information are particularly crucial than others. Prior research [111], [112] identified these neurons and either deactivated or removed them to eliminate the encoded privacy information. Memorized examples can be unlearned by fine-tuning only the high-gradient weights, as demonstrated by Yu et al. [113], who utilized gradient-based saliency to identify and retrain critical model weights responsible for biased behaviors [27]. This is similar to LLMs knowledge-editing technologies, such as that by Meng et al. [114], who modified feedforward weights to update specific factual associations using rank-one model editing (ROME). Building on ROME, MEMIT [115] is a scalable multilayer update algorithm that explicitly computes parameter updates to insert new memories.

2) *Add Extra Trainable Parameter.* This approach is primarily based on the principles of adapter tuning and low-rank adaptation (LoRA) within parameter-efficient fine-tuning techniques. Given that the MLP layers in transformers play a crucial role in representing and storing knowledge, unlearning is achieved by updating the parameters of adapter (LoRA) modules introduced after (within) the MLP layers. Despite its efficiency, this approach may increase inference time and fails to achieve fully unlearning. Removing the extra parameters causes the model to revert to its preunlearning state, thereby violating the RTBF. In [116], a lightweight adapter acts as an unlearning layer within transformers, placed after the MLP layer and trained with a selective teacher–student objective. Meanwhile, Ishibashi and Shimodaira [117] incorporated a single LoRA block into the MLP layer to unlearn harmful information stored within the MLP. Furthermore, Zhang et al. [118] utilized multiple LoRA blocks, each trained on a specific subdataset (task). This method creates combinable modules through linear weight space operations, with a negation operator enabling the unlearning of specific data (task) without further training. Given that LLMs cannot fully access training data, Gao et al. [106] proposed the O3 framework, which includes an Out-Of-Distribution detector to assess input similarity with unlearning data and an Orthogonal LoRA for continuous unlearning. During inference, the O3 framework dynamically loads the unlearning LoRA based on detector predictions.

c) Few-Shot (or Zero-Shot) Unlearning: Unlearning in few-shot (or zero-shot) settings for traditional classification tasks is generally limited. Techniques such as prompt engineering and in-context learning have endowed LLMs with robust few-shot (or zero-shot) capabilities, enabling them to address novel tasks with minimal or no task-specific prompts. This advancement has opened avenues for achieving few-shot or zero-shot unlearning in LLMs, where models can swiftly forget information during inference through the input of minimal unlearning prompts, without necessitating modifications to the model's architecture or parameters. Moreover, this approach also shows promise in addressing challenges related to the limited access to black-box LLMs. However, given that the core model remains unaltered, it continues to preserve the knowledge that was intended to be erased, thereby failing to fulfill the requirements of the RTBF. Muresanu et al. [119] proposed an exact unlearning algorithm, ERASE, which relies on quantized k-means to select few-shot examples that are prepended to the prompt given to LLMs for unlearning. Similarly, Pawelczyk et al. [105] utilized forget samples alongside differently labeled and correctly labeled examples as input prompts to the LLMs during inference. In a recent study, Gallegos et al. [120] introduced a prompting-based bias mitigation method called zero-shot self-debiasing, which leverages the zero-shot capabilities of LLMs to reduce the generation of biased stereotypical expressions. Beyond the aforementioned methods, Fuchi and Takagi [121] introduced a novel few-shot unlearning for text-to-image diffusion models. Their approach updates only the text encoder using a few real images and a prompt, erasing a concept only within 10 s.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

B. Unlearning for Recommender Systems

Recommender systems are personalized information filters that analyze users' preferences from collected data and recommend the most relevant items. Through the training phase, the recommender model's parameters can memorize user behaviors, which poses a risk of privacy leakage. This led to the development of recommendation unlearning, enabling the unlearning of specific data and personal preferences in the model [26].

1) Challenge: Unlearning methods designed for generic ML models engaged in classification tasks are not directly applicable to recommender systems. The reason is that the fundamental principle of most recommender systems is collaborative filtering, but existing MU methods ignore the collaborative information between users and items [95].

2) Methods: Chen et al. [95] utilized the similarity of data to divide training data into balanced groups, based on which they proposed an unlearning algorithm tailored to recommender systems retain the collaborative information within the data while augmenting usability, security, and applicability of the model. Yuan et al. [122] turned their attention to federated recommendation systems; inspired by the log-based rollback mechanism of transactions, they have developed a method that can efficiently retract a user's contribution to the federated training process, thereby enhancing the robustness of the model and strengthening resistance against potential attacks from malicious clients. Since 2023, increasing studies have designed recommendation unlearning methods based on a variety of techniques, including those based on the influence function, matrix completion [123], interaction and mapping matrices correction [26], and even adversarial training [124]. MU promotes the development of more private, secure, reliable, useful, and responsible recommender systems.

C. Unlearning for the Internet of Things

The Internet of Things is a network of interconnected physical devices and objects that can collect and exchange data over the Internet, enabling remote monitoring and control of various applications. IoT service providers frequently face the task of updating DL-based detection models for traffic anomaly detection due to issues like mislabeled samples, device firmware upgrades, or data contamination during service delivery. This demonstrates the urgency of applying MU.

1) Challenge: IoT is usually time-sensitive and requires real-time or near real-time data feedback, so the unlearning process for IoT must be responded to and completed promptly [96].

2) Methods: Fan et al. [96] introduced a method called ViFLa, which groups training datasets according to the calculated unlearning probability, and then each group is regarded as an individual virtual client. ViFLa can improve the effectiveness and completeness of model updates in IoT traffic anomaly detection. Zeng et al. [125] proposed CADDEraser, an effective unlearning framework for Quality-of-Service prediction in personalized IoT, enhancing model utility after unlearning requests. MU can substantially enhance the security, availability, fidelity, and privacy dimensions within IoT systems.

D. Unlearning for Different Sectors

Beyond the key application areas mentioned, MU presents distinct challenges across various sectors, each may require customized approaches. For instance, in finance, models must adhere to stringent privacy regulations while maintaining effectiveness in tasks like credit risk assessment. Exact unlearning in such cases ensures complete data removal without compromising the model's functionality. In social networks, the challenge lies in safeguarding user privacy while preserving community dynamics, as data removal can potentially disrupt relationships. Graph-based unlearning techniques may address this, which maintains the integrity of the network structure. In healthcare, where each hospital's user data is highly sensitive, FL is often employed across multiple institutions to train a global model without sharing patient data. To ensure complete removal of specific data, local-side unlearning can be performed, allowing each hospital to unlearn its downloaded model.

Summary: MU plays a critical role in enhancing privacy, security, and usability across diverse domains like LLMs, recommender systems, and IoT. However, existing approaches are inadequate in addressing the challenges of LLMs unlearning. Most approaches (i.e., FUP and PEU) rely on white-box access to the model or require access to the original training data. Alternatively, while some methods are efficient (i.e., few-shot unlearning), they lack robustness and achieve only superficial forgetting, leaving them vulnerable to the potential threat of sensitive data extraction via white-box attacks.

VII. Attacks on MU

While the dynamic nature of MU is to safeguard the privacy of the data contributor, it might inadvertently expose traces of forgotten data. This could potentially facilitate adversaries' new avenues for attack, making the unlearned model more vulnerable than the original model [126]. This section focuses on attacks specifically targeting MU, categorized into two main types: privacy attacks and security attacks. Privacy attacks lead to privacy breaches, while security attacks degrade model performance and efficiency, or cause the model to generate content that violates human values. We begin by introducing the threat model.

A. Threat Model

1) Adversarial Knowledge: Understanding the degree of adversarial knowledge is critical, as it directly affects the strategies and effectiveness of attacks. Typically, the knowledge an attacker has can be divided into three categories.

1) *White Box:* The adversary has full access to the model's architecture, parameters, and training data, although this level of access is rare in practice.
2) *Gray Box:* The adversary has partial knowledge, such as some access to the model's architecture, parameters, or limited training data.
3) *Black Box:* The adversary knows nothing of the model's structure or parameters. They rely solely on input–output pairs to analyze the model, common in MLaaS scenarios.

2) Attack Stages: Throughout the model's lifecycle, various stages are vulnerable to attack. The primary risks emerge during the following three phases.

1) *Preunlearning (Original Training)*: Adversaries can poison the training data to compromise the original model and prepare for subsequent attacks.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
SUMMARY OF ATTACKS AGAINST UNLEARNING

Attack		Paper	Knowledge			Stage			Attack Vector	Defence
			Wh.[1]	Gr.[2]	Bl.[3]	Pre.[4]	In.[5]	Post.[6]		
Privacy Attacks	MU-MIA[7]	[127]			✓			✓	Model confidence score shifts before and after unlearning	Temperature scaling; Differential privacy; Publishing the label only
		[128]			✓			✓	Model prediction label change before and after unlearning	Differential privacy; Adversarial training; Label smoothing
		[129]			✓			✓	Query comparison between unlearned and original models	Adversarial training; Fine-tuning
	MU-DRA[8]	[130]	✓					✓	Utilize the estimated covariance matrix	Differential privacy
		[129]	✓					✓	Estimate gradient differences before and after unlearning	Adversarial training; Fine-tuning
		[131]	✓					✓	Model weight gap before and after unlearning	N/A
Security attacks	MU-DPA[9]	[126]		✓		✓			Poisoned training dataset	Poisoning detection tool
		[132]			✓	✓			Carefully crafted training dataset	Monitoring the model
		[133]	✓	✓		✓			Carefully crafted noise	poisoning detection tool
	MURA[10]	[134]			✓		✓		Carefully crafted unlearned data (request)	Hash verification; Membership verfication
		[135]	✓	✓	✓		✓		Carefully crafted unlearned data (request)	Hash verification; Membership verification
	MU-JA[11]	[136]			✓			✓	In-context learning	N/A
		[137]	✓					✓	Adversarial perturbations to the embeddings of prompt tokens	Adversarial mechanisms
		[138]	✓					✓	Optimizing adversarial suffixes	Adversarial mechanisms

(1) Wh. – White-box (2) Gr. – Gray-box (3) Bl. – Black-box (4) Pre. – Pre-unlearning (5) In. – In-unlearning (6) Post. – Post-unlearning
(7) MU-MIA – MU-specific Membership Inference Attack (8) MU-DPA – MU-specific Data Poisoning Attack (9) MURA – Malicious Unlearning Request Attack
(10) MU-DRA – MU-specific Data Reconstruction Attack (11) MU-JA – MU-specific Jailbreak Attack

2) *Unlearning:* During this stage, adversaries can send malicious unlearning requests to compromise model performance and efficiency.
3) *Post Unlearning:* Adversaries can access the original (preunlearning) and unlearned models (referred to as the "two models") and exploit their differences to reveal supposedly forgotten information.

B. Privacy Attacks

There are two main types of existing privacy attacks against MU: membership inference attacks and data reconstruction attacks. Generally, attackers exploit the discrepancies between the two models (i.e., original and unlearned models) to infer the membership of the target samples or reconstruct forgotten data.

1) MU-Specific Membership Inference Attack (MU-MIA): Because $\mathcal{D}_u$ often tends to contain more sensitive information than $\mathcal{D}_r$, the goal of the MU-MIA is to determine whether a target sample is forgotten data [127]. Specifically, Chen et al. [127] strategically aggregated posterior scores (confidence scores) generated by models before and after unlearning for a target sample, using these aggregated scores to train attack models capable of inferring membership. However, this attack has a weakness: releasing only predictive labels (top-1 confidence score) effectively mitigates the risk. To address this limitation, Lu et al. [128] injected perturbations into target samples and observed changes in prediction labels between two models to successfully infer membership. Furthermore, Hu et al. [129] extended MU-MIA to LLMs unlearning by comparing queries between the unlearned and original models to determine whether a textual sample is unlearned data.

2) MU-Specific Data Reconstruction Attack (MU-DRA): Research indicates that applying MU on different models can increase the risk of knowledge leakage, as attackers can exploit white-box or gray-box access to the models before and after unlearning to reconstruct relevant information of the unlearned data (i.e., feature and label information, as well as the data itself). Specifically, for MU on linear regression, Bertran et al. [130] effectively recovered deleted samples by leveraging the closed-form training algorithm and estimated covariance matrix. For MU on DNNs, Hu et al. [129] reconstructed the feature and label information of an unlearned sample by estimating the gradient differences before and after unlearning. Similarly, for MU on LLMs, Du et al. [131] reconstructed the unlearned data using only the weight gap between the two models.

3) Defenses: Since both of the aforementioned privacy attacks exploit differences between the original and unlearned models, privacy protection techniques can be employed to mitigate model sensitivity to data. Specifically, DP can be utilized to add random noise to the model's output or gradients, while adversarial mechanisms can introduce carefully designed perturbations to the data during the training phase. Both methods make the model less sensitive to individual data points, preventing attackers from inferring information by comparing the differences between the two models. Additionally, techniques such as label smoothing can smooth the model's predictions, thereby significantly increasing the difficulty for attackers to detect model changes.

C. Security Attacks

There are three main categories of existing security attacks against MU, where data poisoning attacks and malicious unlearning request attacks (MURAs) degrade the ability of the model, and jailbreak attacks make the model produce harmful content.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

1) MU-Specific Data Poisoning Attack (MU-DPA): Attackers strategically poison the training dataset, crafting it in a way that (malicious) effects remain hidden in the original model. However, after unlearning the carefully crafted data, these effects become evident. First, the MU-DPA can serve as a targeted attack, reducing the model's predictive performance on specific samples. For instance, in [126], the training dataset comprises three parts: poisoned data, mitigation data, and normal data. The mitigation data conceal the impact of the poisoned data on the original model, allowing it to evade detection. Attackers then request the unlearning of the mitigation data, thereby unleashing the poisoned data's influence on the model. Ma et al. [132] distilled the data distribution information into a compact set of benign data. These data are recognized as benign by poisoning detection tools because of their positive impact on model training. However, unlearning these data significantly degrades model utility. Second, MU-DPA can decrease the promised unlearning efficiency. For example, Marchant et al. [133] introduced carefully crafted noise to trigger the unlearning process far more than typically required, significantly increasing computational and time costs, akin to traditional denial-of-service attacks.

2) Malicious Unlearning Request Attack: Generally, the attacker requests the model provider to unlearn specially crafted data, causing the model to unlearn more information than expected (normal unlearn requests). Specifically, this type of attack can be untargeted, reducing the model's overall capability after unlearning. As shown in [134], attackers can minimally perturb the data to shift them near or across decision boundaries, leading to a performance decline after unlearning these data. Moreover, such attacks can also cause the unlearned model to misclassify specific data. As discussed in [135], attackers manipulate the decision boundary to selectively cross a target class, causing the model to misclassify certain data into a target class.

3) MU-Specific Jailbreak Attack: As previously discussed, MU can serve as a method of safety alignment to prevent LLMs from generating outputs that conflict with human values. However, recent studies have highlighted a significant vulnerability in current LLMs unlearning techniques: they lack robustness, allowing precisely forgotten knowledge to be reintroduced by MU-specific jailbreak attacks (MU-JAs). This work [136] argues that while unlearning may prevent a model from recognizing a specific concept, the underlying axioms that form this concept are still retained within the model, as these axioms are utilized for other concepts present in the model. By associating the target concept with the underlying axioms through in-context learning, a jailbreak can be achieved. Another common approach is to explore the dependency between the parameters of the unlearned LLMs model and the (embedding of) prompts [137], [138]. Specifically, by introducing adversarial perturbations to the embeddings of prompt tokens or optimizing adversarial suffixes added to the prompt, knowledge that was intended to be erased can be revealed.

4) Defenses: To defend against MU-DPA, it is essential to utilize poisoning detection tools to identify and remove harmful data prior to training. In the case of defending against MURAs, even if service providers in MLaaS settings cannot retain the original training dataset, they can still preserve its hash value. By comparing the hash value of the requested unlearned data and training data or performing membership inference, it is possible to verify whether the data was part of the original training set; if not, the unlearning request should be denied. To defend against MU-JAs, it is crucial to develop more robust LLMs unlearning techniques. For example, Yuan et al. [137] propose enhancing unlearning robustness using adversarial mechanisms.

Summary: Table VI provides a comprehensive summary of various attacks against unlearning and their corresponding defense measures. Although both MU-DPA and MURA degrade model performance, the former injects poisoned data during initial training, whereas the latter submits carefully crafted unlearning requests. Furthermore, while both MU-JA and MU-DRA cause the unlearned model to reproduce forgotten data, they differ fundamentally. The former involves an attacker who is aware of the target knowledge and aims to have the model reproduce it, causing misalignment, while the latter involves an attacker who is unaware of specific knowledge but seeks to extract it, leading to privacy breaches. These attacks underscore the need for more robust unlearning algorithms.

VIII. Challenges and Prospects

To advance reliable MU, this section focuses on key challenges and actionable future directions, providing strategic guidance for future studies. We prioritize challenges and directions with significant practical impact, introducing them in order of importance.

A. More Robust Unlearning for Large Models

The rapid rise of large foundation models (i.e., LLMs and text-to-image models) has profoundly transformed the technological landscape, ushering in a new era further amplified by multimodal LLMs (i.e., GPT-4). However, the critical issues of security and privacy within these models urgently demand the deployment of unlearning techniques. Implementing unlearning algorithms in large models poses significant challenges (as discussed in Section VI). Moreover, existing LLMs unlearning methods prove to be insufficiently robust or lacking guarantees of effective forgetting, making them vulnerable to jailbreak attacks that can reproduce the forgotten knowledge. Incorporating content filtering or adversarial training into the unlearning process may enhance the privacy and security of unlearned models. Developing more robust unlearning algorithms applicable to various large models is the most essential future research direction.

B. Usable Unified Verification Metric

Verification metrics are essential for the practical implementation of MU technologies. However, there is no unified standard for assessing the quality of unlearning, and existing verification metrics are insufficient for several reasons. First, verification metrics should not negatively impact the model; however, some metrics compromise the model's predictive capacity and pose privacy risks. For example, watermark-based metrics may introduce backdoors, thereby diminishing predictive accuracy [34], [37]. Second, verification metrics should be accessible and straightforward for the ordinary user, yet many existing metrics fail in this regard. Cryptographic-based metrics, for instance, present significant challenges to nonexpert users due to their complexity. A feasible approach could involve evaluating the differences in the model's performance on the unlearned data before and after unlearning. Designing a secure, effective, and user-friendly verification metric that can serve as a unified standard requires careful consideration.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

C. Effectiveness and Efficiency DU

Current research on DU is limited, focusing primarily on FL, while other distributed frameworks such as split learning and peer-to-peer learning remain underexplored. Comprehensive challenges as discussed in Section V. Moreover, existing FU methods often fail to balance effectiveness and efficiency, either leaving sensitive information in the global model [85], [91] or causing significant time delays due to server–client interactions [63], [86], [87], [93], [94]. Additionally, since server-side FU may access the unlearned data, it raises further privacy concerns, a viable solution could be for clients to send encrypted unlearning requests to the server. Therefore, developing unlearning techniques that balance effectiveness, efficiency, and privacy across diverse distributed learning is crucial.

D. Unlearning for Features and Tasks

Unlearning algorithms typically target class-based and item-based requests, which are insufficient to fully address user needs. Privacy risks may arise from datasets sharing specific features [62]. Furthermore, since ML models are frequently trained for multiple tasks, it is essential to remove private data associated with specific tasks [32]. However, the sequential unlearning of samples in such contexts presents significant challenges due to high computational costs and the risk of degrading model utility [16]. A promising method for overcoming this challenge could be to explicitly designate which tasks are permanent and which are temporary during training, thereby simplifying their removal postservice [32]. Consequently, unlearning at the feature or task level is crucial to effectively address the diverse requirements encountered in real-world scenarios.

Summary: The outlined challenges represent critical focal points for advancing MU research. Prioritizing robust unlearning in large models and developing a unified verification metric appear to be most urgent, given their significant role in practical implementation. We hope that these promising directions could serve as a beacon for scholars.

IX. Conclusion

In recent years, numerous legal regulations have emerged requiring model (service) providers to promptly and effectively delete users' data and its impact on models in response to their requests. MU is a new technology that can satisfy such as "the right to be forgotten." In this survey, we have presented a comprehensive introduction to MU, providing the basic knowledge for interested scholars. To ensure the practical implementation of MU, we have differentiated between verification and evaluation metrics, systematically summarizing and categorizing each respectively. Moreover, in categories of exact unlearning and approximate unlearning, we provide a more subcategorization based on different underlying strategies used. Additionally, our survey highlights the analysis of MU within the distributed learning setting, particularly, the focused FL. Furthermore, we have pointed out the significant potential applications and summarized specific attacks against MU. Finally, we have underscored current challenges confronting MU and posited auspicious directions for prospective research.

References

[1] A. Ndikumana, N. H. Tran, D. H. Kim, K. T. Kim, and C. S. Hong, "Deep learning based caching for self-driving cars in multi-access edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 5, pp. 2862–2877, May 2021.

[2] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A survey of autonomous driving: Common practices and emerging technologies," *IEEE Access*, vol. 8, pp. 58443–58469, 2020.

[3] J. Jumper et al., "Highly accurate protein structure prediction with AlphaFold," *Nature*, vol. 596, pp. 583–589, Aug. 2021.

[4] J. Cheng et al., "Accurate proteome-wide missense variant effect prediction with AlphaMissense," *Science*, vol. 381, no. 6664, p. 7492, Sep. 2023.

[5] H. Else, "Abstracts written by ChatGPT fool scientists," *Nature*, vol. 613, no. 7944, p. 423, Jan. 2023.

[6] C. Stokel-Walker and R. Van Noorden, "What ChatGPT and generative AI mean for science," *Nature*, vol. 614, no. 7947, pp. 214–216, Feb. 2023.

[7] L. Yang et al., "Diffusion models: A comprehensive survey of methods and applications," *ACM Comput. Surv.*, vol. 56, no. 4, pp. 1–39, 2023.

[8] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.

[9] Z. Ma, Y. Liu, X. Liu, J. Liu, J. Ma, and K. Ren, "Learn to forget: Machine unlearning via neuron masking," *IEEE Trans. Depend. Secure Comput.*, vol. 20, no. 4, pp. 3194–3207, Jul./Aug. 2022.

[10] C. Zhou et al., "PPA: Preference profiling attack against federated learning," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, Jan. 2023.

[11] (2018). *General Data Protection Regulation*. [Online]. Available: https://gdpr-info.eu/

[12] Y. Cao and J. Yang, "Towards making systems forget with machine unlearning," in *Proc. IEEE Symp. Security Privacy (SP)*, May 2015, pp. 463–480.

[13] D. Arpit et al., "A closer look at memorization in deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 233–242.

[14] N. Carlini, C. Liu, U. Erlingsson, J. Kos, and D. Song, "The secret sharer: Evaluating and testing unintended memorization in neural networks," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 267–284.

[15] A. Tahiliani, V. Hassija, V. Chamola, and M. Guizani, "Machine unlearning: Its need and implementation strategies," in *Pro. ACM Int. Conf. Proc. Ser.*, 2021, pp. 241–246.

[16] T. Tam Nguyen et al., "A survey of machine unlearning," 2022, *arXiv:2209.02299*.

[17] H. Zhang, T. Nakamura, T. Isohara, and K. Sakurai, "A review on machine unlearning," *Social Netw. Comput. Sci.*, vol. 4, no. 4, p. 337, Apr. 2023.

[18] N. Romandini, A. Mora, C. Mazzocca, R. Montanari, and P. Bellavista, "Federated unlearning: A survey on methods, design guidelines, and evaluation metrics," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Oct. 25, 2024, doi: 10.1109/TNNLS.2024.3478334.

[19] J. Xu, Z. Wu, C. Wang, and X. Jia, "Machine unlearning: Solutions and challenges," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 3, pp. 2150–2168, Jun. 2024.

[20] Y. Qu, X. Yuan, M. Ding, W. Ni, T. Rakotoarivelo, and D. Smith, "Learn to unlearn: Insights into machine unlearning," *Computer*, vol. 57, no. 3, pp. 79–90, Mar. 2024.

[21] T. Shaik, X. Tao, H. Xie, L. Li, X. Zhu, and Q. Li, "Exploring the landscape of machine unlearning: A comprehensive survey and taxonomy," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Nov. 12, 2024, doi: 10.1109/TNNLS.2024.3486109.

[22] H. Xu, T. Zhu, L. Zhang, W. Zhou, and P. S. Yu, "Machine unlearning: A survey," *ACM Comput. Surv.*, vol. 56, no. 1, pp. 1–36, Jan. 2024.

[23] Y. Cao, A. F. Yu, A. Aday, E. Stahl, J. Merwine, and J. Yang, "Efficient repair of polluted machine learning systems via causal unlearning," in *Proc. Asia Conf. Comput. Commun. Secur.*, May 2018, pp. 735–747.

[24] S. Schelter, "'Amnesia'—Machine learning models that can forget user data very fast," in *Proc. Annu. Conf. Innov. Data Syst. Res.*, 2020.

[25] S. Schelter, S. Grafberger, and T. Dunning, "HedgeCut: Maintaining randomised trees for low-latency machine unlearning," in *Proc. Int. Conf. Manag. Data*, Jun. 2021, pp. 1545–1557.

[26] J. Liu et al., "Recommendation unlearning via matrix correction," 2023, *arXiv:2307.15960*.

[27] A. Blanco-Justicia et al., "Digital forgetting in large language models: A survey of unlearning methods," 2024, *arXiv:2404.02062*.

[28] T. Guo, S. Guo, J. Zhang, W. Xu, and J. Wang, "Efficient attribute unlearning: Towards selective removal of input attributes from feature representations," 2022, *arXiv:2202.13295*.

[29] R. Eldan and M. Russinovich, "Who's Harry potter? Approximate unlearning in LLMs," 2023, *arXiv:2310.02238*.

[30] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

[31] T. Yang, J. Cao, and C. Xu, "Pruning for robust concept erasing in diffusion models," 2024, *arXiv:2405.16534*.

[32] B. Liu, Q. Liu, and P. Stone, "Continual learning and private unlearning," in *Proc. Mach. Learn. Res.*, Jan. 2022, pp. 243–254.

[33] Y. He, G. Meng, K. Chen, J. He, and X. Hu, "DeepObliviate: A powerful charm for erasing data residual memory in deep neural networks," 2021, *arXiv:2105.06209*.

[34] X. Gao et al., "VeriFi: Towards verifiable federated unlearning," 2022, *arXiv:2205.12709*.

[35] C. Zhou, A. Fu, and Z. Dai, "TruVRF: Towards triple-granularity verification on machine unlearning," 2024, *arXiv:2408.06063*.

[36] Y. Li, X. Lyu, N. Koren, L. Lyu, B. Li, and X. Ma, "Anti-backdoor learning: Training clean models on poisoned data," in *Proc. NIPS*, vol. 34, 2021, pp. 14900–14912.

[37] D. Marco Sommer, L. Song, S. Wagh, and P. Mittal, "Towards probabilistic verification of machine unlearning," 2020, *arXiv:2003.04247*.

[38] Y. Guo, Y. Zhao, S. Hou, C. Wang, and X. Jia, "Verifying in the dark: Verifiable machine unlearning by using invisible backdoor triggers," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 708–721, 2024.

[39] Z. Izzo, M. A. Smart, K. Chaudhuri, and J. Zou, "Approximate data deletion from machine learning models," in *Proc. Mach. Learn. Res.*, Jan. 2020, pp. 2008–2016.

[40] X. Cao, J. Jia, and N. Z. Gong, "IPGuard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, May 2021, pp. 14–25.

[41] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 3–18.

[42] M. Kurmanji, P. Triantafillou, and E. Triantafillou, "Towards unbounded machine unlearning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 1957–1987.

[43] L. Graves, V. Nagisetty, and V. Ganesh, "Amnesiac machine learning," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 13, pp. 11516–11524.

[44] D. Ye, H. Chen, S. Zhou, T. Zhu, W. Zhou, and S. Ji, "Model inversion attack against transfer learning: Inverting a model without accessing it," 2022, *arXiv:2203.06570*.

[45] V. S. Chundawat, A. K. Tarun, M. Mandal, and M. Kankanhalli, "Zero-shot machine unlearning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2345–2354, 2023.

[46] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2015, pp. 1322–1333.

[47] A. Salem, A. Bhattacharya, M. Backes, M. Fritz, and Y. Zhang, "Updates-leak: Data set inference and reconstruction attacks in online learning," in *Proc. 29th Secur. Symp. (USENIX Secur.)*, 2020, pp. 1291–1308.

[48] T. Eisenhofer, D. Riepel, V. Chandrasekaran, E. Ghosh, O. Ohrimenko, and N. Papernot, "Verifiable and provably secure machine unlearning," 2022, *arXiv:2210.09126*.

[49] A. Golatkar, A. Achille, A. Ravichandran, M. Polito, and S. Soatto, "Mixed-privacy forgetting in deep networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 792–801.

[50] A. K. Tarun, V. S. Chundawat, M. Mandal, and M. Kankanhalli, "Fast yet effective machine unlearning," 2021, *arXiv:2111.08947*.

[51] L. Bourtoule et al., "Machine unlearning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2021, pp. 141–159.

[52] C. Guo, T. Goldstein, A. Hannun, and L. Van Der Maaten, "Certified data removal from machine learning models," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3832–3842.

[53] A. Golatkar, A. Achille, and S. Soatto, "Eternal sunshine of the spotless net: Selective forgetting in deep networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9301–9309.

[54] S. Neel, A. Roth, and S. Sharifi-Malvajerdi, "Descent-to-delete: Gradient-based methods for machine unlearning," in *Proc. Mach. Learn. Res.*, 2021, pp. 931–962.

[55] A. Sekhari, J. Acharya, G. Kamath, and A. T. Suresh, "Remember what you want to forget: Algorithms for machine unlearning," in *Proc. NIPS*, 2021, pp. 18075–18086.

[56] Y. Wu, E. Dobriban, and S. Davidson, "DeltaGrad: Rapid retraining of machine learning models," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 10355–10366.

[57] J. Brophy and D. Lowd, "Machine unlearning for random forests," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 1092–1104.

[58] A. Golatkar, A. Achille, and S. Soatto, "Forgetting outside the box: Scrubbing deep networks of information accessible from input–output observations," in *Proc. ECCV*, in Lecture Notes in Computer Science, 2020, pp. 383–398.

[59] A. Thudi, G. Deza, V. Chandrasekaran, and N. Papernot, "Unrolling SGD: Understanding factors influencing machine unlearning," in *Proc. IEEE 7th Eur. Symp. Secur. Privacy (EuroS&P)*, Jun. 2022, pp. 303–319.

[60] H. Yan, X. Li, Z. Guo, H. Li, F. Li, and X. Lin, "ARCANE: An efficient architecture for exact machine unlearning," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 4006–4013.

[61] Y. Wu, V. Tannen, and S. B. Davidson, "PrIU: A provenance-based approach for incrementally updating regression models," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Jun. 2020, pp. 447–462.

[62] A. Warnecke et al., "Machine unlearning of features and labels," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2023.

[63] Z. Zhang, Y. Zhou, X. Zhao, T. Che, and L. Lyu, "Prompt certified machine unlearning with randomized gradient smoothing and quantization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 13433–13455.

[64] J. Wu, Y. Yang, Y. Qian, Y. Sui, X. Wang, and X. He, "GIF: A general graph unlearning strategy via influence function," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 651–661.

[65] V. S. Chundawat, A. K. Tarun, M. Mandal, and M. Kankanhalli, "Can bad teaching induce forgetting? Unlearning in deep networks using an incompetent teacher," in *Proc. AAAI Conf. Artif. Intell.*, Jan. 2022, vol. 37, no. 6, pp. 7210–7217.

[66] W. Wang, Z. Tian, C. Zhang, and S. Yu, "Machine unlearning: A comprehensive survey," 2024, *arXiv:2405.07406*.

[67] A. K. Tarun, V. S. Chundawat, M. Mandal, and M. Kankanhalli, "Deep regression unlearning," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2022, pp. 33921–33939.

[68] S. T. Jose and O. Simeone, "A unified PAC-Bayesian framework for machine unlearning via information risk minimization," in *Proc. IEEE 31st Int. Workshop Mach. Learn. Signal Process. (MLSP)*, Oct. 2021, pp. 1–6.

[69] R. Kashef, "A boosted SVM classifier trained by incremental learning and decremental unlearning approach," *Expert Syst. Appl.*, vol. 167, Apr. 2021, Art. no. 114154.

[70] E. Ullah, T. Mai, A. Rao, R. A. Rossi, and R. Arora, "Machine unlearning via algorithmic stability," in *Proc. Mach. Learn. Res.*, Jul. 2021, pp. 4126–4142.

[71] M. Chen, Z. Zhang, T. Wang, M. Backes, M. Humbert, and Y. Zhang, "Graph unlearning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 499–513.

[72] C. Wang, M. Huai, and D. Wang, "Inductive graph unlearning," in *Proc. USENIX Secur. Symp.*, Jan. 2023, pp. 3205–3222.

[73] V. Gupta, C. Jung, S. Neel, A. Roth, S. Sharifi-Malvajerdi, and C. Waites, "Adaptive machine unlearning," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2021, pp. 16319–16330.

[74] M. Chen, W. Gao, G. Liu, K. Peng, and C. Wang, "Boundary unlearning: Rapid forgetting of deep networks via shifting the decision boundary," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 7766–7775.

[75] D. L. Felps et al., "Class clown: Data redaction in machine unlearning at enterprise scale," 2020, *arXiv:2012.04699*.

[76] K. Wu, J. Shen, Y. Ning, T. Wang, and W. H. Wang, "Certified edge unlearning for graph neural networks," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 2606–2617.

[77] D. Yu, H. Zhang, W. Chen, J. Yin, and T. Liu, "How does data augmentation affect privacy in machine learning?" in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 12, pp. 10746–10753.

[78] P. W. Koh and P. Liang, "Understanding black-box predictions via influence functions," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1885–1894.

[79] R. Giordano, W. Stephenson, R. Liu, M. I. Jordan, and T. Broderick, "A Swiss army infinitesimal jackknife," in *Proc. Int. Conf. Artif. Intell. Stat.*, Apr. 2019, pp. 1139–1147.

[80] J. Martens, "New insights and perspectives on the natural gradient method," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 5776–5851, Jan. 2014.

[81] J. Wang, S. Guo, X. Xie, and H. Qi, "Federated unlearning via class-discriminative pruning," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 622–632.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.

[82] A. Thudi, H. Jia, I. Shumailov, and N. Papernot, "On the necessity of auditable algorithmic definitions for machine unlearning," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.)*, 2022, pp. 4007–4022.

[83] L. Wang, T. Chen, W. Yuan, X. Zeng, K.-F. Wong, and H. Yin, "KGA: A general machine unlearning framework based on knowledge gap alignment," 2023, *arXiv:2305.06535*.

[84] L. Zhang, T. Zhu, H. Zhang, P. Xiong, and W. Zhou, "FedRecovery: Differentially private machine unlearning for federated learning frameworks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4732–4746, 2023.

[85] C. Wu, S. Zhu, and P. Mitra, "Federated unlearning with knowledge distillation," 2022, *arXiv:2201.09441*.

[86] Y. Liu, L. Xu, X. Yuan, C. Wang, and B. Li, "The right to be forgotten in federated learning: An efficient realization with rapid retraining," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jun. 2022, pp. 1749–1758.

[87] W. Wang, Z. Tian, C. Zhang, A. Liu, and S. Yu, "BFU: Bayesian federated unlearning with parameter self-sharing," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, Jul. 2023, pp. 567–578.

[88] G. Yu et al., "Split unlearning," 2023, *arXiv:2308.10422*.

[89] A. Fu, X. Zhang, N. Xiong, Y. Gao, H. Wang, and J. Zhang, "VFL: A verifiable federated learning with privacy-preserving for big data in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3316–3326, May 2022.

[90] C. Zhou, A. Fu, S. Yu, W. Yang, H. Wang, and Y. Zhang, "Privacy-preserving federated learning in fog computing," *IEEE Internet Things J.*, vol. 7, no. 11, pp. 10782–10793, Nov. 2020.

[91] G. Liu, X. Ma, Y. Yang, C. Wang, and J. Liu, "FedEraser: Enabling efficient client-level data removal from federated learning models," in *Proc. IEEE/ACM 29th Int. Symp. Quality Service (IWQOS)*, Jun. 2021, pp. 1–10.

[92] J. H. Paik, "A novel TF-IDF weighting scheme for effective ranking," in *Proc. 36th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2013, pp. 343–352.

[93] T. Che et al., "Fast federated machine unlearning with nonlinear functional theory," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 4241–4268.

[94] X. Zhu, G. Li, and W. Hu, "Heterogeneous federated knowledge graph embedding learning and unlearning," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2444–2454.

[95] C. Chen, F. Sun, M. Zhang, and B. Ding, "Recommendation unlearning," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 2768–2777.

[96] J. Fan, K. Wu, Y. Zhou, Z. Zhao, and S. Huang, "Fast model update for IoT traffic anomaly detection with machine unlearning," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8590–8602, May 2023.

[97] Y. Yao, X. Xu, and Y. Liu, "Large language model unlearning," 2023, *arXiv:2310.10683*.

[98] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with CLIP latents," 2022, *arXiv:2204.06125*.

[99] K. Gu, M. Rafi Ur Rashid, N. Sultana, and S. Mehnaz, "Second-order information matters: Revisiting machine unlearning for large language models," 2024, *arXiv:2403.10557*.

[100] C. Barrett et al., "Identifying and mitigating the security risks of generative AI," *Found. Trends Privacy Secur.*, vol. 6, no. 1, pp. 1–52, 2023.

[101] S. Liu et al., "Rethinking machine unlearning for large language models," 2024, *arXiv:2402.08787*.

[102] C. Metz, "Openai says New York times lawsuit against it is 'without merit,"' *New York Times*, 2024.

[103] T. Kaufmann, P. Weng, V. Bengs, and E. Hüllermeier, "A survey of reinforcement learning from human feedback," 2023, *arXiv:2312.14925*.

[104] Y. Zhang et al., "Siren's song in the AI ocean: A survey on hallucination in large language models," 2023, *arXiv:2309.01219*.

[105] M. Pawelczyk, S. Neel, and H. Lakkaraju, "In-context unlearning: Language models as few shot unlearners," 2023, *arXiv:2310.07579*.

[106] C. Gao, L. Wang, C. Weng, X. Wang, and Q. Zhu, "Practical unlearning for large language models," 2024, *arXiv:2407.10223*.

[107] J. Jang et al., "Knowledge unlearning for mitigating privacy risks in language models," 2022, *arXiv:2210.01504*.

[108] K. Chen et al., "Machine unlearning in large language models," 2024, *arXiv:2404.16841*.

[109] L. Wang, X. Zeng, J. Guo, K.-F. Wong, and G. Gottlob, "Selective forgetting: Advancing machine unlearning techniques and evaluation in language models," 2024, *arXiv:2402.05813*.

[110] W. Shi et al., "Detecting pretraining data from large language models," 2023, *arXiv:2310.16789*.

[111] X. Wu et al., "DEPN: Detecting and editing privacy neurons in pretrained language models," 2023, *arXiv:2310.20138*.

[112] N. Pochinkov and N. Schoots, "Dissecting language models: Machine unlearning via selective pruning," 2024, *arXiv:2403.01267*.

[113] C. Yu et al., "Unlearning bias in language models by partitioning gradients," in *Proc. Annu. Meet. Assoc. Comput. Linguist.*, 2023, pp. 6032–6048.

[114] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, Jan. 2022, pp. 17359–17372.

[115] K. Meng, A. Sen Sharma, A. Andonian, Y. Belinkov, and D. Bau, "Mass-editing memory in a transformer," 2022, *arXiv:2210.07229*.

[116] J. Chen and D. Yang, "Unlearn what you want to forget: Efficient unlearning for LLMs," 2023, *arXiv:2310.20150*.

[117] Y. Ishibashi and H. Shimodaira, "Knowledge sanitization of large language models," 2023, *arXiv:2309.11852*.

[118] J. Zhang et al., "Composing parameter-efficient modules with arithmetic operation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 12589–12610.

[119] A. Muresanu, A. Thudi, M. R. Zhang, and N. Papernot, "Unlearnable algorithms for in-context learning," 2024, *arXiv:2402.00751*.

[120] I. O. Gallegos et al., "Self-debiasing large language models: Zero-shot recognition and reduction of stereotypes," 2024, *arXiv:2402.01981*.

[121] M. Fuchi and T. Takagi, "Erasing concepts from text-to-image diffusion models with few-shot unlearning," 2024, *arXiv:2405.07288*.

[122] W. Yuan, H. Yin, F. Wu, S. Zhang, T. He, and H. Wang, "Federated unlearning for on-device recommendation," in *Proc. 16th ACM Int. Conf. Web Search Data Mining*, Feb. 2023, pp. 393–401.

[123] M. Xu, J. Sun, X. Yang, K. Yao, and C. Wang, "Netflix and forget: Efficient and exact machine unlearning from bi-linear recommendations," 2023, *arXiv:2302.06676*.

[124] C. Ganhör, D. Penz, N. Rekabsaz, O. Lesota, and M. Schedl, "Unlearning protected user attributes in recommendations with adversarial training," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2022, pp. 2142–2147.

[125] Y. Zeng, J. Xu, Y. Li, C. Chen, Q. Dai, and Z. Du, "Towards highly-efficient and accurate services QoS prediction via machine unlearning," *IEEE Access*, vol. 11, pp. 76242–76254, 2023.

[126] J. Z. Di, J. F. Douglas, J. Acharya, G. Kamath, and A. Sekhari, "Hidden poison: Machine unlearning enables camouflaged poisoning attacks," in *Proc. NeurIPS ML Saf. Workshop*, 2023, pp. 44424–44450.

[127] M. Chen, Z. Zhang, T. Wang, M. Backes, M. Humbert, and Y. Zhang, "When machine unlearning jeopardizes privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 896–911.

[128] Z. Lu, Y. Wang, Q. Lv, M. Zhao, and T. Liang, "FP^2-MIA: A membership inference attack free of posterior probability in machine unlearning," in *Proc. Int. Conf. Provable Secur.*, 2022, pp. 167–175.

[129] H. Hu, S. Wang, T. Dong, and M. Xue, "Learn what you want to unlearn: Unlearning inversion attacks against machine unlearning," 2024, *arXiv:2404.03233*.

[130] M. Bertran, S. Tang, M. Kearns, J. Morgenstern, A. Roth, and Z. S. Wu, "Reconstruction attacks on machine unlearning: Simple models are vulnerable," 2024, *arXiv:2405.20272*.

[131] J. Du, Z. Wang, and K. Ren, "Textual unlearning gives a false sense of unlearning," 2024, *arXiv:2406.13348*.

[132] B. Ma et al., "Releasing malevolence from benevolence: The menace of benign data on machine unlearning," 2024, *arXiv:2407.05112*.

[133] N. G. Marchant, B. I. Rubinstein, and S. Alfeld, "Hard to forget: Poisoning attacks on certified machine unlearning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 7, 2022, pp. 7691–7700.

[134] H. Hu et al., "A duty to forget, a right to be assured? Exposing vulnerabilities in machine unlearning services," 2023, *arXiv:2309.08230*.

[135] C. Zhao, W. Qian, R. Ying, and M. Huai, "Static and sequential malicious attacks in the context of selective forgetting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 74966–74979.

[136] I. Shumailov et al., "UnUnlearning: Unlearning is not sufficient for content regulation in advanced generative AI," 2024, *arXiv:2407.00106*.

[137] H. Yuan, Z. Jin, P. Cao, Y. Chen, K. Liu, and J. Zhao, "Towards robust knowledge unlearning: An adversarial framework for assessing and improving unlearning robustness in large language models," 2024, *arXiv:2408.10682*.

[138] L. Schwinn, D. Dobre, S. Xhonneux, G. Gidel, and S. Gunnemann, "Soft prompt threats: Attacking safety alignment and unlearning in open-source LLMs through the embedding space," 2024, *arXiv:2402.09063*.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.



Na Li received the bachelor's degree from Xinjiang University, Ürümqi, China, in 2023. She is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Nanjing University of Science and Technology, Nanjing, China.

Her research interests include AI privacy and security.



Chunyi Zhou received the Ph.D. degree from the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China, in 2024, supervised by Prof. Anmin Fu.

He is currently a Post-Doctoral Researcher collaborating with Prof. Shouling Ji at the College of Computer Science and Technology, Zhejiang University, Hangzhou, China. His research interests include AI privacy preserving and security, federated learning, and machine unlearning.



Yansong Gao (Senior Member, IEEE) received the M.Sc. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2013, and the Ph.D. degree from The University of Adelaide, Adelaide, SA, Australia, in 2017.

He is a Lecturer at The University of Western Australia, Perth, WA, Australia. His current research interests include AI security and privacy, hardware security, and system security.

Dr. Gao serves as an Associate Editor for IEEE TRANSACTIONS ON FORMATION FORENSICS AND SECURITY and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.



Hui Chen received the bachelor's degree in industrial engineering from Nanjing University of Science and Technology, Nanjing, China, in 2023, where she is currently pursuing the master's degree with the School of Computer Science and Engineering.

Her research interests include AI privacy and security.



Zhi Zhang (Member, IEEE) received the bachelor's degree from Sichuan University, Chengdu, China, in 2011, the master's degree from Peking University, Beijing, China, in 2014, and the Ph.D. degree in computer science from The University of New South Wales, Kensington, NSW, Australia, in 2021.

He is a Lecturer at the University of Western Australia, Perth, WA, Australia. His research interests include system security, Rowhammer, and adversarial artificial intelligence.



Boyu Kuang received the Ph.D. degree in cyber space security from Nanjing University of Science and Technology, Nanjing, China, in 2023.

He is currently a Post-Doctoral Researcher at Nanjing University of Science and Technology. His research interests include the IoT security and vulnerability mining.



Anmin Fu (Member, IEEE) received the Ph.D. degree in information security from Xidian University, Xi'an, China, in 2011.

He is currently a Professor at Nanjing University of Science and Technology, Nanjing, China. He has authored or co-authored more than 100 technical articles, including international journals and conferences, such as IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE Symposium on Security and Privacy, ACM Conference on Computer and Communications Security (ACM CCS), Network and Distributed System Security Symposium (NDSS), Asia Conference on Computer and Communications Security (Asia CCS), and Australasia Conference on Information Security and Privacy (ACISP). His research interests include the IoT security, cloud computing security, and privacy preserving.

Authorized licensed use limited to: Bangladesh Univ of Engineering and Technology. Downloaded on January 30,2026 at 13:01:47 UTC from IEEE Xplore. Restrictions apply.